

摘要：

中泰证券认为，算力互联演进主线并非简单“以光替铜”，而是将光电转换位置从前面板逐步移向芯片端。从可插拔光模块到NPO、CPO，产业链价值正向高功率激光器、FAU、先进封装等环节重新分配。

AI推理需求的爆发式增长，正在把数据中心互联推向一个新的临界点。

8月29日，中泰证券分析师王芳、孙悦文发布研报《光学向芯片靠近：AI 算力互联的演进主线从铜互联、可插拔光模块到NPO与CPO》认为，AI推理Token需求爆炸式增长，Google两年内月处理量从9.7万亿增至超3200万亿，倒逼算力互联从铜缆向近封装光学加速演进。而技术演进主线不是简单地“用光替代铜”，更重要的是“逐步把光电转换位置从前面板移向ASIC附近，以减少越来越难以补偿的高速电信号传输”。

从铜互联、可插拔光模块，到NPO（近封装光学）、CPO（共同封装光学），每一步技术迭代背后，都是同一个驱动力：**高速电信号越来越难传，光学就得越来越靠近芯片**。

产业链价值将从传统可插拔光模块向高功率CW激光器、FAU、高密度连接、先进封装等环节重新分配，但业绩兑现仍需以客户定点和批量交付为依据。

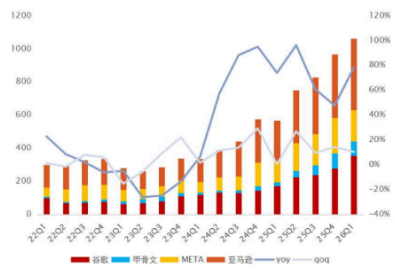
为什么互联成了算力瓶颈？

分析师援引Google数据称，其各产品端月度处理量由2024年5月的9.7万亿增至2025年5月约480万亿，并在2026年5月超过3,200万亿，**两年增长超过300倍**。

Token量暴增的直接后果是：GPU之间需要更频繁、更快速地协同工作。分析师指出，“Token增加会显著提高多卡协同的重要性，GPU越快、专家数量越多、推理并发越高，系统越需要低时延、高带宽的全互连网络。”

资本开支数据印证了这一判断。截至2026年二季度，Google将全年资本开支预期提升至1,950亿—2,050亿美元，Amazon预计约2,200亿美元，Meta预计1,300亿美元。

图表 2：北美 CSP 季度资本开支（单位：亿美元）



来源：WIND，中泰证券研究所

图表 3：国内 CSP 季度资本开支（单位：亿元）



来源：WIND，中泰证券研究所

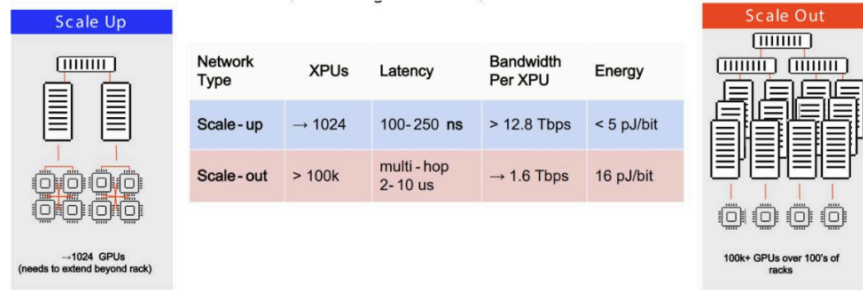
与此同时，互联架构也在分化为两条主线。分析师区分了Scale-up与Scale-out的不同角色：

Scale-up把一组XPU组织成低时延、紧耦合的计算域，强调单芯片注入带宽、尾时延、确定性流控以及集体通信效率；

Scale-out把更多服务器、机柜和计算域连接起来，强调网络规模、跨域吞吐、路由弹性和开放互操作。

两者都需要光互联，但对功耗、故障边界和维修方式的要求并不相同。

图表 5: Scale-up 与 Scale-out 在规模、时延、带宽和能耗目标上存在明显差异



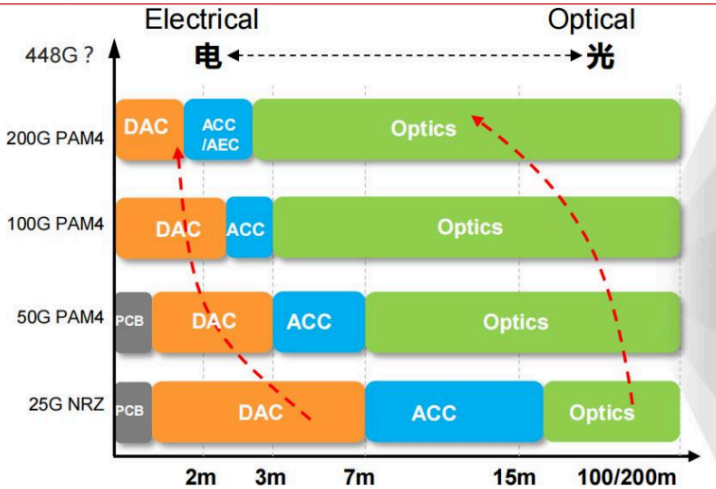
来源：OCP，中泰证券研究所

铜的边界在哪里？

单通道速率的提升，正在压缩铜互联能覆盖的距离。

分析师援引华为数据指出，随着SerDes由25G NRZ向50G、100G、200G PAM4演进，PCB、DAC等纯电互联的有效距离持续缩短。Broadcom在特定场景中估算，100Gbps/lane DAC约可覆盖4米，提升至200Gbps/lane后可能缩短至2米；OIF对224G/lane接口给出的无源铜目标则集中在约1米。

图表 6: 单通道速率提升持续压缩 PCB、DAC 与 ACC/AEC 的覆盖范围

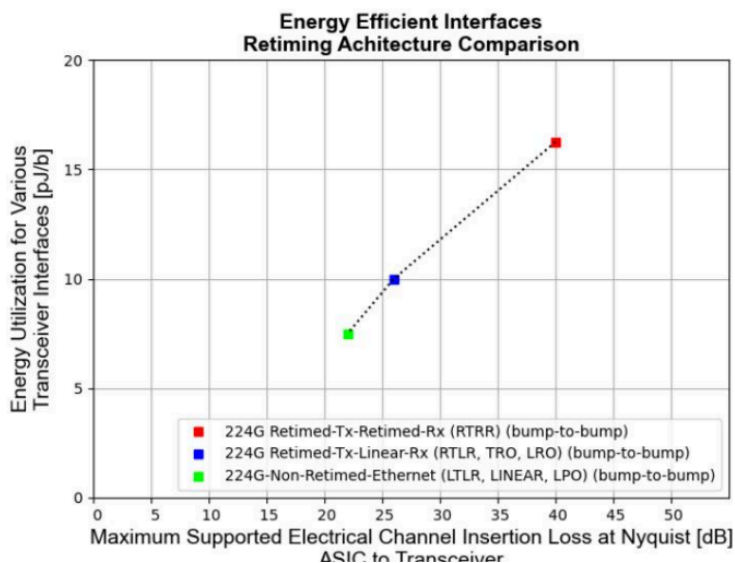


来源：华为，中泰证券研究所

铜的退出不是一刀切。分析师强调，“铜转光不是统一时点，而是不同距离层级依次重新计算经济性”——封装内和板级连接厘米到米级，机架内跨越数米到数十米，叶脊网络和数据中心互联距离更长，光纤低损耗和标准化的优势才更加突出。

可插拔光模块是铜向光演进的第一次大规模产品化，但它的瓶颈也在显现。分析师指出，“交换容量每翻倍一次，若继续依靠相同粒度的可插拔端口，模块数量、DSP功耗和前面板散热同步上升”。有限的面板空间无法同时容纳更多端口和更大的散热结构。

图表 8：电通道损耗越高，补偿能耗通常越大



来源：OIF，中泰证券研究所

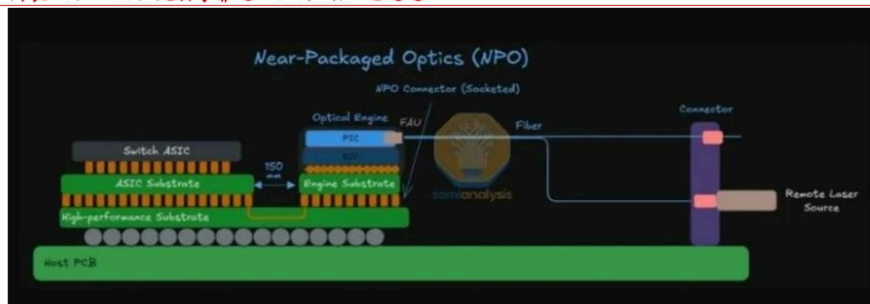
NPO：过渡平台，不是终点

NPO（Near-Packaged Optics，近封装光学）的核心逻辑，是把光引擎从前面板“搬”到ASIC旁边，但保留可分离的边界。

分析师解释：“NPO把可独立制造、测试和更换的光引擎部署在ASIC附近，在缩短电通道的同时，保留了部件筛选、良率隔离和一定的维修弹性。”

这带来两项直接收益：一是降低电通道损耗，二是良率解耦——“高价值ASIC、光引擎、FAU和外置光源可以分别形成Known Good Die或合格子组件，再在系统装配阶段组合；某颗光引擎异常时，可分离设计有机会只更换失效部件，避免把局部光学缺陷放大为整套ASIC封装报废。”

图表 10：NPO：光引擎靠近 ASIC，缩短电通道



来源：Semianalysis，中泰证券研究所

链路损耗数据更直观。分析师援引OIF在112G/lane示例中的数据：前面板可插拔的bump-to-bump损耗约为22—24dB，NPO约13dB，CPO约10dB。“NPO已经通过缩短主板通道获得大部分改善。”

图表 12: XPU 至 NPO 链路的插入损耗模型

损耗环节	106G（奈奎斯特26.6GHz）	128G（奈奎斯特32.0GHz）
GPU基板损耗（5mm）	0.65 dB	0.70 dB
Host PCB损耗（60mm）	5.40 dB	6.00 dB
BGA	0.04 dB	0.04 dB
连接器	0.50 dB	0.50 dB
铜柱（×2）	0.04 dB	0.04 dB
总通道损耗	6.6 dB	7.3 dB

来源：Photonics，中泰证券研究所

但NPO并不简单。分析师指出，其产业化难点“并不局限于光引擎本身，还包括224G高速电接口、PIC与EIC封装、外置光源、高通道FAU、板内光纤、冷却以及管理测试等系统接口”。

光源的位置尤为关键。外置ELS/ELSFP把激光器放在较低温且可更换的位置，但“增加保偏光纤、连接器、分光和耦合损耗，需要更高纤耦输出功率及更完整的光功率闭环”。

在技术路线上，硅光与VCSEL并行发展，并非非此即彼。分析师认为，“短距、高并行、光纤数量可接受的场景可以优先评估VCSEL；需要WDM、更长覆盖或更高前面板带宽密度的系统更适合硅光。设备商也可能在同一超节点内为不同链路层级采用不同光学平台。”

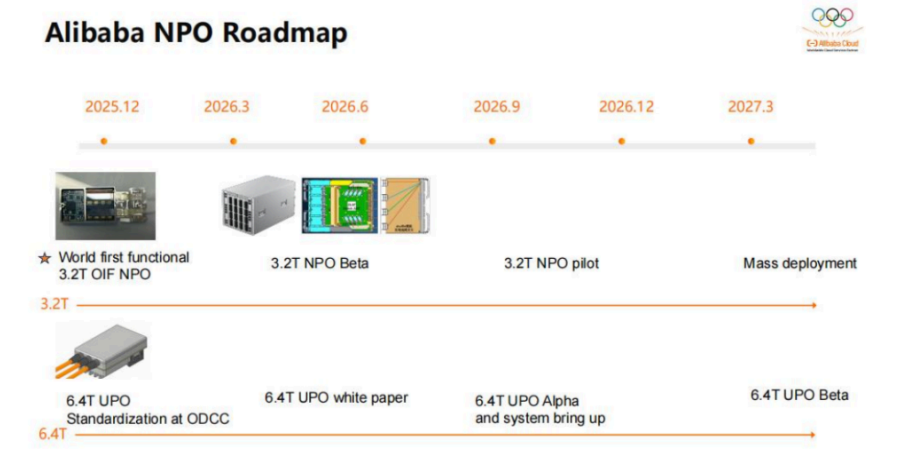
图表 23: 硅光 NPO 与 VCSEL NPO 的适用边界

比较维度	硅光 NPO	VCSEL NPO
光源与调制	CW激光器 + MZM/微环等调制器	VCSEL阵列直接调制
光纤与复用	单模光纤，适合WDM	多模光纤，通常并行一纤一路
核心优势	高带宽密度	阵列与光学结构相对简单，制造成熟、成本潜力较低
典型场景	多机架 Scale-up、高带宽 Scale-out	AI Scale-up 中的短距/多机架多模互连

来源：Broadcom，中泰证券研究所

从产业进展看，阿里已公开以3.2T NPO作为试点起点，并同步推进6.4T UPO与接口标准化；腾讯则同时验证3.2T硅光和VCSEL链路；华为Hi-ONE平台同时布局VCSEL与硅光路线，并把光引擎、外置光源、连接、冷却和链路监测放入同一系统框架。

图表 25: 阿里公开路线由 3.2T NPO 试点向 6.4T UPO 与标准化演进



来源：阿里巴巴，中泰证券研究所

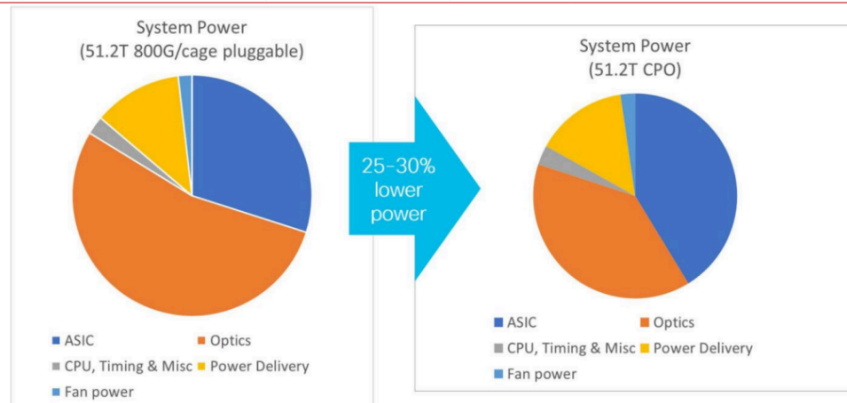
分析师强调，“当前3.2T NPO主要承担试点和运行数据积累功能，向6.4T、7.2T升级则意味着电气、光学、封装和测试体系整体换代”。NPO能否从样品走向通用平台，“最终取决于接口标准、装配良率、长期运行数据和现场维修成本”。

CPO：更深的集成，更高的门槛

CPO（Co-Packaged Optics，共封装光学）把交换ASIC与光引擎纳入共同封装，把高速电路径压缩至毫米级或封装级。

功耗收益可量化。分析师援引Cisco在特定51.2T系统口径下的估算：CPO相较可插拔架构可降低约25%—30%的整机功耗。Broadcom在特定576 GPU互联场景中给出的数据更具体：DSP可插拔、LRO和CPO光学功耗分别约为16.2kW、11.7kW和7.1kW。

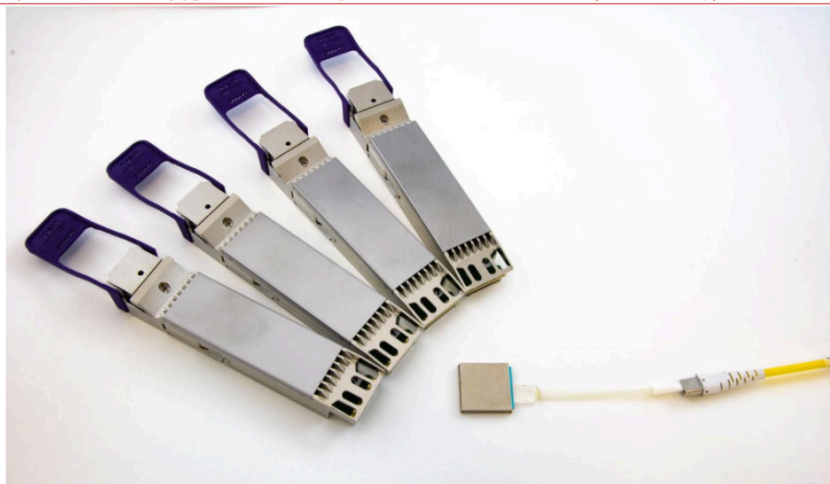
图表 28：特定 51.2T 系统示例中，CPO 通过减少光学与冷却负担降低整机功耗



来源：Cisco，中泰证券研究所

面板密度的变化同样直观。分析师指出，“四只800G OSFP模块能够提供3.2T带宽，一只3.2T CPO光引擎也能提供相同数量级的带宽，但后者不再需要在前面板保留完整模块外壳、笼子和模块散热器。”

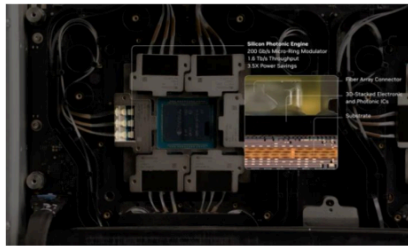
图表 29：相同 3.2T 带宽下，完整可插拔模块（4x800G）与 CPO 光引擎占用空间差异明显



来源：Cisco，中泰证券研究所

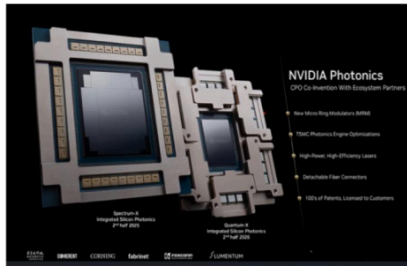
英伟达是目前CPO落地最具代表性的案例。分析师详述：英伟达已落地Quantum-X800-Q3450和Spectrum-X两代CPO交换机，分别用于InfiniBand及以太网。Quantum-X整机搭载4颗Quantum-3交换ASIC，整机聚合总带宽115.2Tbps，整机含72颗光引擎OE。Spectrum-X中，单OE带宽提升至3.2Tbps，整机有效带宽 $32 \times 3.2T = 102.4Tbps$ 。

图表 31: Quantum-X (InfiniBand) 内部结构



来源: Nvidia, 中泰证券研究所

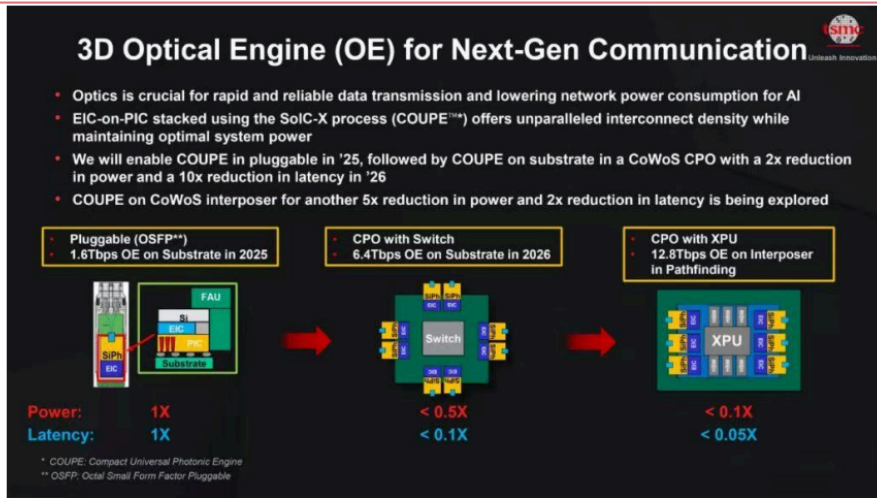
图表 32: Spectrum-X Photonics (以太网)



来源: Nvidia, 中泰证券研究所

台积电COUPE平台是支撑CPO规模化的关键制造基础。分析师指出, COUPE“正逐渐成为CPO/OIO主流选择方案”——英伟达在GTC 2025大会上展示了COUPE光引擎, 成为首批上市的COUPE产品; 博通未来技术路线图也已从其原有供应链转向COUPE平台; Ayar Labs也将COUPE纳入发展规划。

图表 33: 台积电的 COUPE 正成为主流选择



来源: TSMC, 中泰证券研究所

但量产门槛不低。分析师列出五道门槛: 联合良率、热管理、外置光源、测试诊断、维修。

其中良率是核心。“CPO把高价值ASIC、PIC、EIC、光纤耦合结构和先进封装放入同一产品边界, 任何一处缺陷都可能放大报废损失。”联合良率并非各部件良率的简单相乘, “大尺寸基板、中介层、微凸点和多材料堆叠在回流焊与热循环中会产生翘曲和热膨胀失配, 光纤耦合对微米级位移高度敏感。”

维修问题同样不可回避。“CPO若缺少可分离组件, 局部光学缺陷可能演变为整块交换板或共同封装返厂。CPO必须证明节省的功耗和空间足以覆盖更高的故障处置成本。”

图表 42: 从 NPO 到 CPO, 光互连向更高集成度演进

	DPO/LPO	NPO	CPO	OIO
带宽密度	50~100 Gbps/mm ²	100~200 Gbps/mm ²	200~500 Gbps/mm ²	500~1000 Gbps/mm ²
延迟	20~50 ns	10~20 ns	5~10 ns	<5 ns
能耗	10~20 pJ/bit	8~12 pJ/bit	5~8 pJ/bit	<5 pJ/bit
灵活性	极高 支持热插拔	中 OE与主板绑定	低 OE与芯片绑定	极低 OE与芯粒集成
接口	多种接口	LR SerDes 接口	XSR SerDes 接口	UCle 接口
应用场景	中长距	中短距	中短距	片间直连

来源: 奇安信, 中泰证券研究所

谁受益？产业链价值如何重分配？

光学内移不是光模块价值量的平移，而是“功能位置、器件规格和制造责任同时重构”。

分析师梳理了几个关键增量方向：

高功率CW激光器是外置光源路线中相对明确的增量。集中供光可以减少激光器颗数，却提高单颗功率、可靠性和ELS系统价值。古河电工公开的8通道ELS样机在55°C壳温下实现每通道20dBm纤耦输出。分析师提醒，“研究应关注单位光源输出功率、冗余比例、ELS价值量和客户定点，而不能直接把端口数量映射为激光器颗数”。

保偏光纤需求集中在外置光源至偏振敏感硅光调制器之间的供光链。但分析师特别指出，Intel OCl光I/O芯粒公开采用SMF-28且不需要PMF，“进一步说明保偏光纤具有明确的路线依赖，不能外推为所有CPO出纤的普遍变化”。

FAU和高密度连接的增量逻辑“更具普适性”。光纤进入ASIC周围的高热流、高密度区域后，需在大尺寸基板翘曲、热循环、振动和污染环境下维持微米级对准和低插损，由此带动V形槽、透镜或微光学、MT插芯、MPO/MTP、光纤带、盲插连接器等整条链条升级。

先进封装、测试和热管理的价值将在量产阶段进一步放大。NPO需要高性能有机基板、LGA或插座、夹持结构；CPO更依赖大尺寸基板、中介层或嵌入式桥、微凸点、低翘曲材料。

分析师最后强调，产业链价值重分配的逻辑清晰，但“最终业绩兑现仍需以客户定点、批量交付、良率和可靠性数据为依据”。

图表 43：NPO/CPO 架构变化向新增器件与工艺环节传导



来源：OIF, NVIDIA, Broadcom, TSMC, ASE, 中泰证券研究所

图表 44：CPO/NPO 上游关键器件及材料厂商

公司	代码	对应环节
天孚通信	300394.SZ	FAU、ELS外置光源等CPO配套产品
罗博特科	300757.SZ	光电子及半导体自动化封装、耦合和测试设备
炬光科技	688167.SH	CPO用微透镜、微透镜阵列、光纤耦合等微光学器件
致尚科技	301486.SZ	MTP/MPO、FA-MT、保偏光纤阵列、模场转换光纤阵列等
光库科技	300620.SZ	DR4/DR8 FAU、MT-FAU、保偏光纤阵列、保偏尾纤、WDM、MPO/MTP等
腾景科技	688195.SH	精密光学元器件、光纤阵列、WDM等光通信产品
长飞光纤	601869.SH	保偏光纤
长盈通	688143.SH	CPO极细径抗弯保偏光纤
太辰光	300570.SZ	MTP/MPO高密度光纤连接器、光柔性板、MT插芯等
源杰科技	688498.SH	70mW、100mW等大功率硅光CW光源；更高功率CW光源
仕佳光子	688313.SH	CPO高通量FAU；硅光用CW DFB激光器

来源：公司官网、公司公告，中泰证券研究所

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时权益申领

* 体验资格每人限申领一次

扫码

VIP会员·7天畅读卡 | 大师课·入门串讲

免费领取



限免

151 收藏

分享到:

写评论

请发表您的评论

表情 图片

发表评论

摘要：

从行业维度看，电气机械器材、计算机通信电子设备等行业的生产指数与新订单指数均高于53.0%，产需保持较快释放。新动能行业表现持续强劲。装备制造业和高技术制造业PMI分别为51.4%和52.9%，继续位于扩张区间。

中国制造业景气度在8月出现明显改善，但整体经济活动仍维持在荣枯线以下。

国家统计局8月31日公布的数据显示，8月制造业采购经理指数（PMI）升至49.8%，较上月上升0.6个百分点，为近期景气水平的明显回暖。

值得关注的是，制造业生产指数和新订单指数双双重返扩张区间，价格指数亦明显反弹，出厂价格指数时隔数月重返荣枯线以上。国家统计局服务业调查中心首席统计师霍丽慧表示，8月制造业行业景气度明显改善，在调查的21个行业中有16个行业PMI较上月上升。

从行业维度看，**电气机械器材、计算机通信电子设备等行业的生产指数与新订单指数均高于53.0%，产需保持较快释放**。新动能行业表现持续强劲。**装备制造业和高技术制造业PMI分别为51.4%和52.9%，继续位于扩张区间。**

原材料价格的上行为制造业价格指标带来明显提振。受近期原油及有色金属价格上涨影响，制造业主要原材料购进价格指数和出厂价格指数分别升至56.6%和50.4%，较上月分别大幅上升3.4个和2.6个百分点。

从企业规模看，大型企业PMI为50.6%，比上月上升1.1个百分点，高于临界点；中型企业PMI为49.4%，比上月下降0.3个百分点，低于临界点；小型企业PMI为47.9%，比上月上升0.5个百分点，低于临界点。

与此同时，非制造业方面，8月商务活动指数维持49.0%，与上月持平，整体保持基本稳定。综合PMI产出指数录得49.5%，较上月上升0.2个百分点。

据中国物流与采购联合会副会长何辉介绍，8月份制造业国内外市场需求均有所扩张，**新订单指数和新出口订单指数同比和环比均有明显上升**，市场需求环比变化情况好于去年同期，显示政策端加力，有效增强了经济发展动力。

综合来看，受高温、台风、暴雨洪涝等极端天气的影响，8月制造业采购经理指数仍处在50%以下，但随着扩内需政策较好发力，中央预算内投资的引导和带动效应进一步显现，109项重大工程和“六张网”规划建设进入提速阶段，再加上暑期消费稳步释放以及国际市场需求上升等多个因素带动，指数较上月明显回升，显示出制造业运行回稳，景气水平明显提高。

制造业采购经理指数回升

8月份，制造业采购经理指数（PMI）为49.8%，比上月上升0.6个百分点，景气水平回升。

从企业规模看，大型企业PMI为50.6%，比上月上升1.1个百分点，高于临界点；中型企业PMI为49.4%，比上月下降0.3个百分点，低于临界点；小型企业PMI为47.9%，比上月上升0.5个百分点，低于临界点。

从分类指数看，构成制造业PMI的5个分类指数中，生产指数、新订单指数和供应商配送时间指数均高于临界点；原材料库存指数、从业人员指数低于临界点。

生产指数为50.4%，比上月上升0.5个百分点，表明制造业生产活动有所加快。

新订单指数为50.6%，比上月上升2.1个百分点，表明制造业市场需求明显改善。

原材料库存指数为48.1%，比上月下降0.2个百分点，表明制造业主要原材料库存量继续下降。

从业人员指数为48.7%，比上月下降0.3个百分点，表明制造业企业用工景气度有所回落。

供应商配送时间指数为50.1%，比上月上升0.6个百分点，表明制造业原材料供应商交货时间加快。

表1 中国制造业PMI及构成指数（经季节调整）

单位：%

	PMI	生产	新订单	原材料 库存	从业人员	供应商 配送时间
2025年8月	49.4	50.8	49.5	48.0	47.9	50.5
2025年9月	49.8	51.9	49.7	48.5	48.5	50.8
2025年10月	49.0	49.7	48.8	47.3	48.3	50.0
2025年11月	49.2	50.0	49.2	47.3	48.4	50.1
2025年12月	50.1	51.7	50.8	47.8	48.2	50.2
2026年1月	49.3	50.6	49.2	47.4	48.1	50.1
2026年2月	49.0	49.6	48.6	47.5	48.0	49.1
2026年3月	50.4	51.4	51.6	47.7	48.6	49.5
2026年4月	50.3	51.5	50.6	49.3	48.8	49.5
2026年5月	50.0	51.2	49.9	48.6	48.6	49.2
2026年6月	50.3	51.4	51.2	48.4	48.5	49.9
2026年7月	49.2	49.9	48.5	48.3	49.0	49.5
2026年8月	49.8	50.4	50.6	48.1	48.7	50.1

表2 中国制造业PMI其他相关指标情况（经季节调整）

单位：%

	新出口 订单	进口	采购量	主要原材料 购进价格	出厂 价格	产成品 库存	在手 订单	生产经营 活动预期
2025年8月	47.2	48.0	50.4	53.3	49.1	46.8	45.5	53.7
2025年9月	47.8	48.1	51.6	53.2	48.2	48.2	45.2	54.1
2025年10月	45.9	46.8	49.0	52.5	47.5	48.1	44.5	52.8
2025年11月	47.6	47.0	49.5	53.6	48.2	47.3	45.5	53.1
2025年12月	49.0	47.0	51.1	53.1	48.9	48.2	46.0	55.5
2026年1月	47.8	47.3	48.7	56.1	50.6	48.6	45.1	52.6
2026年2月	45.0	45.6	48.2	54.8	50.6	45.8	44.0	53.2
2026年3月	49.1	49.8	50.9	63.9	55.4	46.7	47.1	53.4
2026年4月	50.3	50.1	51.1	63.7	55.1	47.5	46.2	54.5
2026年5月	48.6	48.8	49.8	60.5	51.9	49.3	46.8	53.9
2026年6月	50.1	49.6	51.4	54.2	48.2	47.7	47.1	54.3
2026年7月	49.6	47.5	49.4	53.2	47.8	48.6	45.8	54.1
2026年8月	50.1	48.6	50.5	56.6	50.4	48.4	46.7	53.8

非制造业商务活动指数与上月持平

8月份，非制造业商务活动指数为49.0%，与上月持平。



分行业看，建筑业商务活动指数为46.9%，比上月下降0.1个百分点；服务业商务活动指数为49.3%，与上月持平。从服务业行业看，邮政、电信广播电视及卫星传输服务、互联网软件及信息技术服务等行业商务活动指数均位于55.0%以上较高景气区间；批发、零售、资本市场服务等行业商务活动指数均低于临界点。

新订单指数为44.1%，比上月下降0.3个百分点，表明非制造业市场需求景气水平有所下降。分行业看，建筑业新订单指数为42.4%，比上月上升2.3个百分点；服务业新订单指数为44.5%，比上月下降0.7个百分点。

投入品价格指数为51.1%，比上月上升1.4个百分点，表明非制造业企业用于经营活动的投入品价格总体水平回升。分行业看，建筑业投入品价格指数为51.2%，比上月上升2.5个百分点；服务业投入品价格指数为51.1%，比上月上升1.2个百分点。

销售价格指数为49.3%，比上月上升1.4个百分点，仍低于临界点，表明非制造业企业销售价格总体水平降幅收窄。分行业看，建筑业销售价格指数为49.8%，比上月上升2.1个百分点；服务业销售价格指数为49.2%，比上月上升1.3个百分点。

从业人员指数为45.4%，与上月持平，低于临界点，表明非制造业企业用工景气度仍然偏弱。分行业看，建筑业从业人员指数为43.0%，比上月上升2.1个百分点；服务业从业人员指数为45.8%，比上月下降0.4个百分点。

业务活动预期指数为55.0%，比上月下降0.4个百分点，仍位于临界点以上，表明非制造业企业对市场发展保持乐观。分行业看，建筑业业务活动预期指数为51.8%，与上月持平；服务业业务活动预期指数为55.5%，比上月下降0.5个百分点。

表3 中国非制造业主要分类指数（经季节调整）

单位：%						
	商务活动	新订单	投入品价格	销售价格	从业人员	业务活动预期
2025年8月	50.3	46.6	50.3	48.6	45.6	56.2
2025年9月	50.0	46.0	49.0	47.3	45.0	55.7
2025年10月	50.1	46.0	49.4	47.8	45.2	56.1
2025年11月	49.5	45.7	50.4	49.1	45.3	56.2
2025年12月	50.2	47.3	50.2	48.0	46.1	56.5
2026年1月	49.4	46.1	50.0	48.8	46.1	56.0
2026年2月	49.5	45.2	50.9	48.8	46.0	55.0
2026年3月	50.1	45.0	52.3	49.9	45.2	54.2
2026年4月	49.4	44.3	51.7	48.1	45.5	54.7
2026年5月	50.1	45.0	52.2	48.8	45.6	54.8
2026年6月	50.2	48.0	49.7	48.4	45.8	55.3
2026年7月	49.0	44.4	49.7	47.9	45.4	55.4
2026年8月	49.0	44.1	51.1	49.3	45.4	55.0

表4 中国非制造业其他分类指数（经季节调整）

单位：%				
	新出口订单	在手订单	存货	供应商配送时间
2025年8月	48.8	43.4	45.7	51.3
2025年9月	49.8	44.4	44.9	51.1
2025年10月	46.2	43.6	46.0	50.9
2025年11月	47.9	43.6	44.8	51.2
2025年12月	47.5	43.5	45.2	51.3
2026年1月	46.9	42.5	44.8	51.1
2026年2月	44.7	42.3	45.4	50.5
2026年3月	48.8	43.2	43.5	51.5
2026年4月	47.3	42.4	45.3	50.9
2026年5月	48.1	43.8	45.1	51.2
2026年6月	47.9	44.2	45.3	51.3
2026年7月	47.0	43.3	45.1	50.9
2026年8月	47.0	42.8	45.6	51.2

综合PMI产出指数有所回升

8月份，综合PMI产出指数为49.5%，比上月上升0.2个百分点，我国企业生产经营景气水平总体有所回升。

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时权益申领

* 体验资格每人限申领一次

扫码 免费领取

VIP会员·7天畅读卡 | 大师课·入门串讲



限免

251 收藏

分享到:

写评论

请发表您的评论



摘要：

高盛最新报告力挺腾讯，混元Hy4预览版参数量暴增2.6倍，编码能力排名从全球第28位跃升至第6位，实现代际跨越。更关键的是，腾讯将模型与WorkBuddy、CodeBuddy深度绑定，在大规模真实用户场景中收集数据，再将这些数据反哺至后续的预训练和后训练迭代，形成正向数据飞轮。

高盛认为，混元Hy4预览版的发布标志着该模型家族迄今最大幅度的代际能力跃升，并将腾讯独特的"产品+模型闭环"策略视为其在AI Agent时代建立差异化竞争优势的核心路径。

据追风交易台，高盛在8月31日发布的研究报告中指出，混元Hy4预览版的发布时间早于分析师预期，延续了自Hy3预览版以来约两个月一次的发布节奏。在编码能力方面，Hy4预览版在Code Arena WebDev全球排行榜上跃升至第6位，而Hy3仅排名第28位，提升幅度尤为突出。

高盛分析师Ronald Keung、Lincoln Kong等人认为，**这一进展将成为腾讯未来12个月股价的重要驱动因素之一**，并维持12个月基于SOTP方法的目标价港币670元，较当前股价港币455.20元隐含约47.2%的上行空间。

尽管资本开支强度上升及AI长期投资将对近期利润增长形成压制——该行预计腾讯2026年三季度及四季度每股盈利同比增速分别仅为4%和0%——但混元模型的持续迭代、WorkBuddy用户指标的强劲表现，以及微信小微的逐步推广，将共同支撑腾讯的中期估值逻辑。

参数规模大幅扩张，编码能力实现代际跨越

混元Hy4预览版在模型规模上实现显著扩张，总参数量从Hy3的2950亿/210亿激活参数，增至7700亿总参数/490亿激活参数，增幅约2.6倍；上下文长度亦从256K tokens扩展至100万tokens。

腾讯将此次发布定性为混元模型家族有史以来最大幅度的代际能力提升，预训练与后训练均经历了实质性规模扩张。在编码能力方面，Hy4预览版在Code Arena WebDev全球排行榜排名第6，较Hy3的第28位大幅跃升，使混元重新跻身开源模型第一梯队。

在专业任务覆盖面上，Hy4预览版在软件工程、办公生产力、游戏开发及科学研究等多个领域均实现了有意义的能力提升。腾讯通过与内部专家团队合作，利用专有领域知识构建围绕真实工作流的训练数据，并与WorkBuddy、CodeBuddy等工作区产品协同设计，以获取有机用户反馈用于模型迭代。

成本效率维持竞争优势，架构创新支撑推理效率

尽管模型规模大幅扩张，Hy4预览版在成本效率方面仍保持竞争力，其混合定价约为每100万tokens 0.45美元，在同等规模的开源同类产品中处于较低水平，且性能表现相当甚至更优。

在架构层面，腾讯引入多项推理效率创新，包括受DeepSeek和GLM启发的门控DSA注意力机制（Gated DSA Attention）、用于提升长上下文计算效率的IndexCache，以及用于增强层间信息流的iHC（identity Hyper-Connections）。上述架构创新有助于腾讯在持续扩大模型规模的同时，维持对同类开源模型的成本优势。

"产品+模型闭环"构筑差异化护城河



报告中着重强调腾讯的差异化策略——预览版模型优先通过腾讯自有生态及AI原生应用（WorkBuddy、CodeBuddy）分发，在大规模真实用户场景中收集数据，再将这些数据反哺至后续的预训练和后训练迭代，形成正向数据飞轮。

这一闭环方法在生产力和编码工作负载场景中尤具价值，因为真实任务轨迹、用户交互及评估信号，正是AI Agent时代模型差异化的核心驱动力。腾讯云及智慧产业集团管理层近期在媒体采访中亦重申，腾讯AI战略日益聚焦于通过持续投入产品、场景和用户采用来创造长期价值，致力于解决真实世界问题并建立持久差异化。

腾讯将于9月1日参加高盛亚洲领袖会议的炉边对话，投资者关注焦点预计将集中于算力资源分配与折旧压力的平衡方式、混元的差异化模型策略、WorkBuddy的规模化路径、微信小微的推广进展，以及游戏和AI驱动广告的前景。

~~~~~  
以上精彩内容来自[追风交易台](#)。

更详细的解读，包括实时解读、一线研究等内容，请加入 [【追风交易台·年度会员】](#)

# 追风

RISK CHASE

全球宏观研究 | 交易 | 配置

你 / 不 / 能 / 错 / 过

## 什么是“追风交易台”

追风交易台由顶流全球宏观资讯及研究团队联手打造。

我们的目标是给中国投资者提供专业、独到和适时的全球金融市场研究。

## 在这里，你将看到

### 最专业的解读

来自全球顶尖投行、研究机构、  
一线从业者的最新解读



### 最独到的视角

以全球市场第一线的视角  
感知市场动向



### 最适时的时机

快速、完整、准确  
最终汇集成适时



## 如果你

关心全球宏观经济和资本市场

## 如果你

苦恼于没有合适的信息和研究





关注“追风交易台”  
成为“追风会员”

你将身处全球市场第一线

公众号 · 追风交易台

风险提示及免责声明

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时权益申领

\* 体验资格每人限申领一次

扫码  
免费领取

VIP会员·7天畅读卡 | 大师课·入门串讲



限免

84 收藏

分享到:

写评论

请发表您的评论

表情 图片

发表评论

摘要：

黄金8月大涨约10%，有望创今年1月以来最大单月涨幅，导火索是美财长贝森特宣布大幅增购长期国债。但与年初相比，这轮行情中投资者明显更为谨慎——他们转向看涨价差期权和奇异期权，而非直接押注上涨，隐含波动率和看涨溢价均低于一季度水平。部分投资者预计金价将在4900至5300美元区间内震荡。

黄金8月累计上涨约10%，有望创下今年1月以来最大单月涨幅。但与年初那轮狂热相比，这一次的衍生品市场明显更为冷静——据彭博最新报道，投资者不再一味买入看涨期权，而是转向价差策略和奇异期权，用更低的成本押注金价继续走高。

这轮行情的导火索，是美国财政部长贝森特宣布计划“至少翻倍”购买10至30年期流通国债。这一举措压低了美元，推升了黄金及其数字替代品比特币。投资者对硬资产的需求在美元购买力持续受侵蚀的背景下明显升温。

不过，美联储主席沃什在杰克逊霍尔会议上重申抗通胀立场，令市场加息预期升温，黄金在周五出现回调，为这轮涨势踩下了刹车。

现货黄金 (XAUUSD)

4438.10 -16.13 (-0.36%)

交易中, 2026-08-31 09:11:00 (北京时间)

+加自选

概览 图表



## 押注方式变了：从“直接买涨”到“精打细算”

与年初相比，投资者对黄金的信心更趋保守。

年初，特朗普公开表示不担心美元贬值，点燃了一轮黄金急涨行情。彼时衍生品市场情绪高度亢奋，被业内称为贵金属领域的“波动率末日”(Volmageddon)。

这一次不同。据彭博报道，交易员大量买入SPDR黄金ETF的看涨价差期权，而非直接买入看涨期权；奇异期权同样受到追捧。两者的共同点是：成本更低，对上行空间的预期也更为克制。

State Street Investment Management全球黄金与金属策略主管Aakash Doshi表示：“投资者通过ETF需求和衍生品市场重新做多黄金。货币贬值交易从未消亡，只是暂停了，如今它正在重新流行。”

他同时指出：“8月的这波行情，远比今年1月贵金属市场的价格波动和衍生品活动更为有序。”

## 波动率低了，预期区间窄了

黄金期权隐含波动率虽有所上升，但仍远低于一季度高点。看涨期权的溢价（即偏斜度）也比年初更窄。

据报道，美国银行奇异期权与流量交易欧中东非区域负责人、全球混合交易联席主管Neeraj Chaudhary表示：“与年初不同的是，黄金波动率相对较低，部分投资者认为下一波上涨的空间更为有限，金价可能维持区间震荡，比如在4900至5300美元之间。”

在具体操作上，双重数字期权（dual-digital options）等奇异结构颇受欢迎。Chaudhary介绍，黄金与货币对的组合交易是热门策略之一——通过外汇端来降低期权成本。

“比如，一些投资者交易了黄金/美元兑日元组合——你可以在黄金上涨、美元上涨的情景下，以接近负20%的相关性买入。”他说。

他还补充道：“我们也看到了一些三元二元期权的需求——比如同时押注黄金、原油和外汇，这能给投资者带来比通常目标10至20倍更高的杠杆收益。”

美国银行股票衍生品结构欧中东非区域主管Joseph Khouri则表示：“过去几个月，黄金双重数字期权一直是主导性的交易流，黄金通常作为跨资产组合中的看涨端。原因在于，黄金的投资逻辑并不依赖单一宏观结果，在多种情景下黄金都可能上涨。”

## 比特币同步飙升，但性质存疑

黄金并非唯一受益于弱美元的资产。比特币自8月19日以来大涨12%，一度突破8万美元，终结了数月的横盘僵局。

Coinglass数据显示，8月19日至21日期间，超过25亿美元的比特币空头头寸在永续合约市场遭到强制平仓。这场空头挤压将一轮宏观驱动的初步上涨放大为更为剧烈的突破行情，同时也将资金重新引入美国上市的现货比特币基金——自8月19日以来，这类基金已吸引逾20亿美元流入。

但市场仍存在一个核心疑问：比特币究竟是在被当作黄金一样的持久宏观对冲工具，还是这不过是一场由杠杆和动量放大的仓位行情？随着空头出清、获利了结开始进行，下一阶段的走势或许更多取决于新增现货需求能否持续跟进，而非被动的强制买盘。

### 风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时权益申领

\* 体验资格每人限申领一次

VIP会员·7天畅读卡 | 大师课·入门串讲

扫码

免费领取



限免

267 收藏

分享到:

### 写评论

请发表您的评论



表情



图片

发布评论

3 条评论



MobileUser2026

4小时前 中国陕西省

8 月 31 日当日下午跌超 %4!



0



回复



见闻用户

6小时前 中国上海

嘴炮无敌😏



0



回复



手机用户

6小时前 中国山东省

美债已经无法与黄金匹配信誉安全。



2



回复





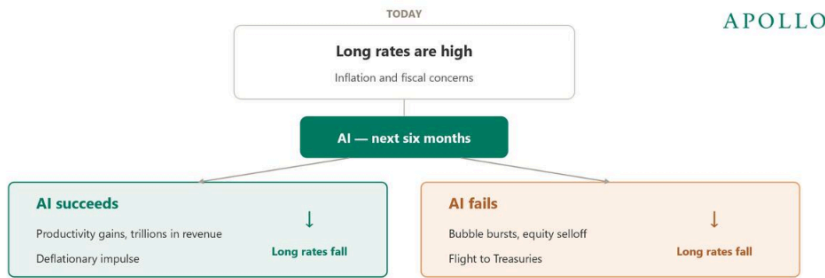
摘要：

Torsten Slok认为，AI商业化成功→大规模通缩效应→利率下行；AI泡沫破裂→纳斯达克暴跌50%→避险资金涌入美债→收益率同样急跌。未来六个月是关键窗口，市场叙事将从"通胀+财政赤字"切换至AI成败，当前高利率或已处于转折前夜。

当前主导美债市场的通胀与财政叙事，正面临一场来自AI的叙事替代。阿波罗全球管理公司首席经济学家Torsten Slok警告，未来六个月内，长端利率大幅下行的风险正在上升——无论AI最终走向成功还是失败，这一结果都难以避免。

Torsten Slok在8月31日的最新报告中指出，AI将成为主导2027年利率走势的核心叙事。若AI商业化成功，科技公司创造数万亿美元营收，其带来的大规模通缩效应将压低长端利率；若AI泡沫破裂，纳斯达克指数可能暴跌50%，投资者将大规模从股市撤离转入美债避险，同样推动长端收益率急剧下行。

If AI succeeds, rates will be lower. If AI fails, rates will be lower.



这一判断的核心逻辑在于：金融市场由叙事驱动。当前利率市场的叙事围绕通胀高企与财政赤字问题展开，但随着时间推移至2027年前后，这一叙事将被AI的成败所取代，而两种结局均指向同一个方向——利率下行。

## AI成功：通缩压低利率；AI失败：泡沫破裂触发避险资金涌入美债

Torsten Slok指出，当前长端利率处于高位，根本原因在于两大结构性压力：持续的通胀问题以及美国财政状况的恶化。这两股力量共同支撑了市场对长期高利率的预期，并主导着当前债券市场的定价逻辑。

然而，Torsten Slok认为，这一叙事框架并非永久性的。他明确表示，上述力量在2027年初有可能被AI的发展走向所压倒，届时市场的注意力将发生根本性转移。

在AI成功的情景下，Torsten Slok描绘了一幅清晰的传导路径：若科技公司凭借AI技术创造出数万亿美元的营收规模，AI将对整体经济产生大规模通缩效应。

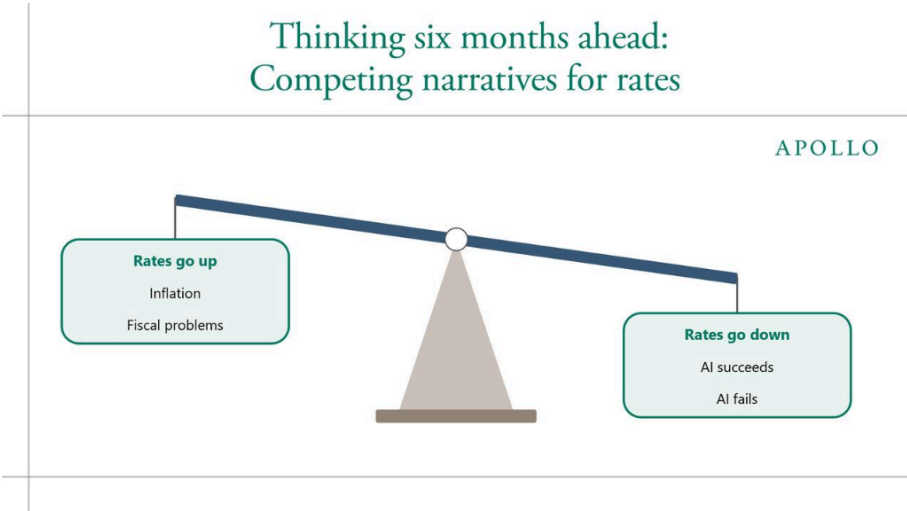
通缩意味着价格压力的系统性下降，这将直接削弱市场对长期通胀的预期，进而推动长端利率走低。在这一情景中，AI的生产力红利转化为价格下行压力，从根本上改变利率市场的定价基础。

若AI发展未能兑现预期，Torsten Slok预判市场将面临泡沫破裂的冲击。他具体指出，纳斯达克指数在这一情景下可能下跌50%，投资者将大规模从股票市场撤出，转而涌入美国国债寻求避险。

资金的大规模轮动将直接推高美债价格、压低收益率，长端利率将出现“急剧下行”。这一逻辑与历史上多次风险资产抛售期间美债的避险属性高度吻合——股市的崩溃反而成为债券市场的强力催化剂。

六个月窗口期：市场将作出判断

Torsten Slok强调，未来六个月是关键观察窗口。在此期间，市场将逐步形成对AI发展路径的判断——究竟是成功兑现商业价值，还是泡沫开始松动。



他指出，金融市场本质上由叙事驱动。当前的利率叙事聚焦于通胀与财政问题，但进入2027年，主导叙事将切换至AI的成败。无论哪种叙事最终胜出，其对长端利率的影响方向是一致的：下行。

这意味着，对于债券投资者而言，当前的高利率环境或许正处于叙事转换的前夜，而提前布局长端美债的逻辑正在悄然积累。

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

97 收藏

分享到:

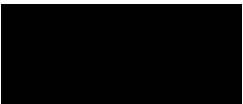
写评论

请发表您的评论

表情 图片

发表评论

2 条评论







摘要：

Salesforce允许客户按业务成果付费——不只计量任务，而是“帮你赚20美元才收2美元”；OpenAI、Sierra等也跟进“任务完成后付费”模式。但成果定价暗藏归因风险：业绩究竟该归功于AI还是营销？这场定价实验将决定老牌软件巨头能否在AI浪潮中完成自我重塑。

人工智能正在颠覆软件行业沿用逾二十年的订阅收费逻辑。以Salesforce为代表的企业软件巨头，正被迫从按用户数收取固定订阅费的模式，转向按使用量乃至按实际业务成果收费——这一转变既是机遇，也充满变数。

据科技媒体The Information 8月30日报道，Salesforce首席执行官Marc Benioff上周在投资者电话会议上表示，公司正允许企业客户自主选择其AI产品Agentforce的付费方式，包括签订定制化合同，按AI帮助销售人员促成更多交易所带来的收入增长，或通过自动化客服所节省的成本来计费。

“客户希望以不同方式购买和定价，这是我最近深刻认识到的，”Benioff说。这番表态折射出AI时代软件定价模式的深层不确定性。

**这一转变已在行业内引发连锁反应。**据报道，知情人士透露，OpenAI近几个月已开始向部分大客户提供仅在AI完成任务后才付费的选项；**客户管理初创公司Sierra和Fin**（Salesforce正以36亿美元收购后者）同样采用任务完成后才收费的模式；**编程助手Cognition**则承诺，若未能为企业客户交付至少等值于付费金额的工程成果，将提供最高1000万美元的抵扣额度。

Salesforce股价自上周财报发布后已累计上涨约23%。



赛富时

NYSE: CRM

256.00 USD

↑ 22.83%

+47.58 过去 5 天

+ 关注

收盘时间: 8月28日 GMT-4下午7:59 • 免责声明

盘后价 256.48 +0.48 (0.19%)



## 订阅模式走向终结

Salesforce此番转型，标志着其亲手缔造的软件即服务（SaaS）商业模式正面临根本性挑战。

二十五年前，正是Salesforce引领了软件行业从一次性买断授权向按员工数量收取订阅费的历史性转变。这一模式降低了中小企业的前期成本，并将软件升级维护的负担转移至供应商一侧，由此开启了SaaS行业长达二十年的繁荣周期。

然而，**AI的崛起正在将这一逻辑倒置**。随着企业越来越多地使用Anthropic的Claude等先进AI智能体处理涉及Salesforce等应用的复杂任务，员工与这些应用直接交互的频率下降，按用户席位收费的订阅模式随之失去根基。

Benioff坦承，软件定价正处于一个充满不确定性的时期，而Salesforce正在跟随初创公司的步伐，而非引领变革。

## 向"成果定价"靠拢

Salesforce目前正在探索的新模式，与数据分析软件公司Palantir长期以来的做法高度相似。

**Palantir与企业客户签订高度定制化合同，将固定费用与基于使用量及业务成果的收费相结合。**

Benioff表示，这种灵活定价方式已帮助Salesforce“签下了非常大额的交易”，并指出供应商在这一模式下可以为产品“获得极高的定价”——Palantir过去一年收入的大幅增长印证了这一判断。

Benioff进一步阐释了其“对‘成果定价’的理解层次：

"我们不只是想完成这么多通电话，所以收2美元，我们希望能够说‘我们帮你提升了这么多收入，所以收2美元，因为我们帮你赚了20美元或40美元’。"

这意味着Salesforce的目标是将自身收益与客户业务成果深度绑定，而非仅停留于任务完成层面的计量收费。

## Claudeforce：新战场上的新布局

面对Anthropic等AI原生竞争者的压力，Salesforce上周推出了Claudeforce——一项允许客户直接使用Claude完成大量涉及Salesforce应用的任务、而无需直接操作这些应用的服务。

据报道，知情人士透露，Salesforce计划通过Claudeforce构建一套收益机制：**每当第三方AI调用Salesforce应用内的数据时，Salesforce均可从中获益；客户需升级至更高订阅层级方可启用该功能。**

这一布局的战略意图在于，即便用户不再直接使用Salesforce界面，公司仍能在AI生态中保持数据层面的核心地位，将潜在的用户流失风险转化为新的变现入口。

## 归因之争：新模式的潜在隐患

报道称，成果定价模式在理论上具有吸引力，但在**实际执行层面可能引发复杂的归因争议**。

支付服务商Stripe已就此发布指引，明确指出销售转化或其他业务成果“可能源于产品调整、营销活动或季节性因素”，而非软件本身的贡献。

"除非归因规则明确，否则客户可能会就成果是否应归功于软件供应商产生争议，"Stripe表示。

**这一隐患并非没有先例**。软件监控公司Splunk在从授权模式向订阅模式转型期间，曾经历收入的阶段性下滑。分析认为，当前这场定价模式的实验，**其结果将在很大程度上决定Salesforce等老牌企业软件公司能否在AI浪潮的冲击下完成自我重塑。**

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时权益申领

\* 体验资格每人限申领一次

扫码  
免费领取

VIP会员·7天畅读卡 | 大师课·入门串讲



限免

71    收藏

分享到:

写评论

请发表您的评论

表情    图片

发表评论

摘要:

英伟达芯片因为原有设计和庞大的公共缓存, 导致处理少量数据时特别卡。OpenAI的新芯片决定推倒重来: 直接砍掉多余的公共缓存, 让计算核心和内存“一对一”直连, 并换用更灵活的计算方式。它从根本上消除了卡顿, 而不是靠堆算力掩盖。更牛的是, 他们让AI来主导芯片和软件设计, 仅9个月就造出实物, 效率彻底碾压对手。

## TL;DR

假如我重生成某基模型团队的 Infra 负责人, 那么一定会对 Nvidia 的 GPU 及整个系统有各种不满, 然后自己做芯片...

对于 Nvidia 网络部分就不用谈了, RoCE各种缺陷, 然后 DPU 也有一堆性能和安全问题, 也难怪今年 Nvidia 又开始吹 Scale-in 了, 但是一个从来没做过云的团队这方面想成熟起码还要 5 年. 举个例子, 最近我们做了一件事情, 在我们 MaaS 在线业务中的某个 PD 分离的场景中, 将传输协议用了 CIPU 的 eRDMA 替换直接把 TTFT 降低了 40%, 直接等同于维持 TTFT SLA的情况下可以省大量的 Prefill 计算服务器...

对于 GPU 微架构, 去年就对此详细写了一篇《[Inside Nvidia GPU: 谈谈Blackwell的不足并预测一下Rubin的微架构](https://github.com/zartbot/blog/issues/3)》(英文版: <https://github.com/zartbot/blog/issues/3>) 批评 Nvidia Blackwell微架构的各种缺陷, 老黄还在 NV 内部转发过...

当时建议 SM 增加用于调度的超标量核, 这不 OpenAI Jalapeño 就有了. 另一个问题是我一直给 Nvidia 在提, 你们 GPU 架构中, 大量的 L2 Cache 占用了太多芯片面积, 但是似乎对它又很难去做一些控制, 缓存层次结构比较厚, 为了维持 UMA 实质上在 L2 两个 Partition 上会增加 200~400 cycle 的延迟, 而且对外 I/O 的交互也无法直接写到 SMEM 内, 这下好了 OpenAI Jalapeño 直接砍掉 L2 Cache. 还有前几个月在卷 Anthropic 的一个面试题, 实际上也是在准备和自研训练和推理芯片相关的任务, 并且在内网也详细分析了各种加速器架构以及该怎么做...

今天这篇分析 Jalapeño 的文章, 我假设我自己是某基模芯片团队的负责人, 从模型的需求以及当前 Nvidia GPU 的缺陷出来, 逐步从第一性原理开始推导该如何设计, 当然也会在公司内部逐渐复现整个设计过程. 通过这种方式来阐述 Jalapeño 的芯片架构以及背后的一些工具链... 本文目录如下:

1. 从 Nvidia GPU 谈起
  - 1.1 推理场景下的理论峰值
  - 1.2 架构决定 E2E 延迟
  - 1.3 Nvidia GPU 的缺陷
2. 如何设计推理芯片
  - 2.1 推理芯片第一性原理
  - 2.2 推理 Infra 现状
    - 2.2.1 不同阶段的性能需求
    - 2.2.2 KVCache 的代价
  - 2.3 推理芯片体系结构
    - 2.3.1 编程接口的变化
    - 2.3.2 体系结构的取舍
3. OpenAI Jalapeño 芯片架构
  - 3.1 芯片架构 Overview
    - 3.1.1 Core Slice 架构
    - 3.1.2 NOC 架构
  - 3.2 Core Slice 架构详解
  - 3.3 内存子系统
  - 3.4 片上网络
  - 3.6 系统架构
4. 软件架构
5. 芯片设计
  - 5.1 关键前提: 先换语言, 再让 AI 上场
  - 5.2 设计如何闭环
6. 未来展望
  - 6.1 对比 Nvidia GPGPU 架构
  - 6.2 OpenAI roadmap
  - 6.3 一些分析总结

⚠ 很多更详细的评估以及复现整个设计流程的工作就不公开了... 由于微信公众号排版不适合电脑前阅读, 因此在 Github 上放了一个重新排版的

中文版: <https://zartbot.github.io/blog/arch/jalapeno/index.html>

## 1. 从 Nvidia GPU 谈起

首先想起的是一个段子, 也是现状.

产品管理的兄弟: 隔壁NV做了, 为啥咱不做?

也是产品管理的兄弟: 隔壁NV都没做, 为啥我们能做?

还是产品管理的兄弟: 隔壁NV做了, 为啥我们还要做?

依旧是产品管理的兄弟: 隔壁NV做了, 我们怎么敢做?

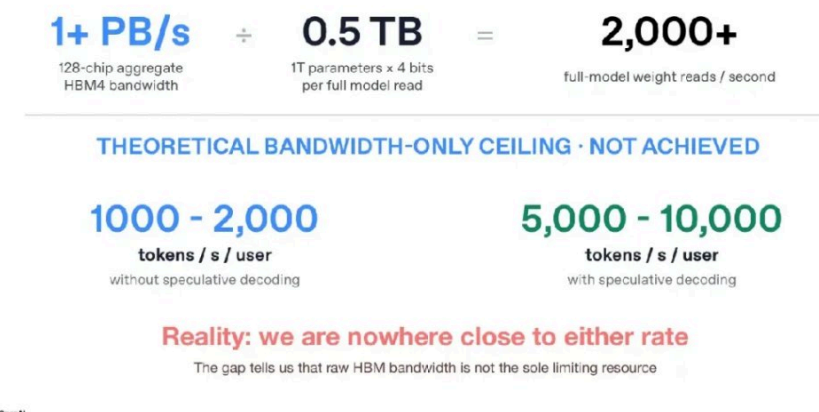
大多数做加速器的厂商, 满脑子的是抄 NV 作业, 然后美其名曰 CUDA 生态兼容, 但再怎么抄整体落后 NV 两到三代... 另外一个极端就是总想搞一些奇奇怪怪的体系结构, 试图弯道超车.... 然而几乎所有的芯片厂商和 NVidia 最大的差距在于全栈的能力, 从模型结构和算法, 再到软件生态, 最后到芯片实现, 其中哪些地方可以有 trade-off, 每次交易赢了什么输了什么, 很少有人能够讲清楚. 恰好我从算法到芯片全栈都懂, 因此可以完整的来阐述整个芯片设计的过程以及 Nvidia GPU 的缺陷.

### 1.1 推理场景下的理论峰值

GPU一直以来“靠大量并发 warp 隐藏访存延迟”的机制处理大量的数据, 这个机制本质上依赖大 batch. Batch 小了, 可调度的 warp 就少, 流水线填不满, 于是远离 roofline. 这便是推理场景中经常遇到的问题, 延迟隐藏机制失效.

既然推理阶段很大程度上是 Memory bound 的, OpenAI 做了一个很简单的计算, 假设累计 ScaleUP 域有 N 个芯片, 每个芯片 HBM 带宽为 M, 估计一个模型参数规模为 K, 那么可以简单的假设计算没有任何瓶颈时, 理论上输出 tokens 的最大速度为

### Raw HBM bandwidth implies extraordinary token rates



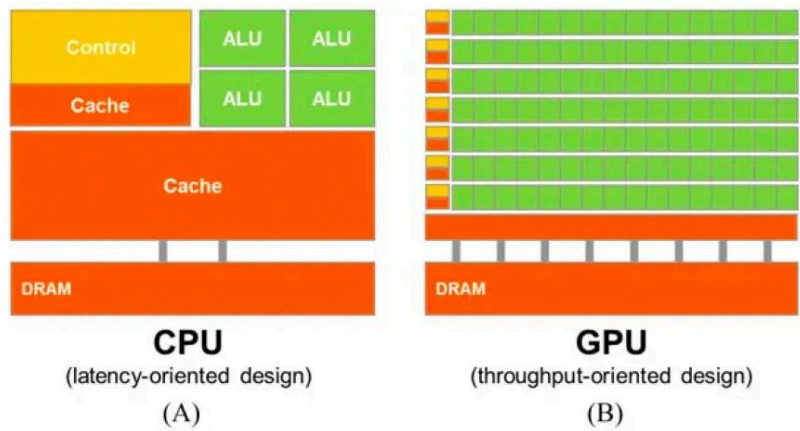
也就是说理论上, 不加投机解码, 1T 参数的模型应该能过做到 1000~2000

tokens/s/user. 但现阶段大多数推理框架在B300一类的平台上, 做多只能 100~200

tokens/s/user, 只有 10% 的理论性能. 因此这一页在说: 就算把 HBM 带宽用到理论极限, 得到的延迟仍然有一个下界. 也就是说, 单靠加带宽解决不了低延迟问题. 既然带宽不是唯一变量, 那么一定是体系结构上的问题了...

### 1.2 架构决定E2E延迟

在一年多前一篇文章里我暗示过, 从处理器的 workload 来看, 在现代计算机体系结构中, CPU的核心设计哲学与GPU截然不同: GPU通过海量并行线程掩盖延迟以最大化吞吐量, 而CPU则是不惜耗费巨大的芯片面积与晶体管功耗预算, 穷尽一切微架构手段将单线程执行延迟 (Single-Thread Latency) 压缩到极致.



CPU通过分支预测提前探路、多发射与乱序执行见缝插针地并行计算, 依托多级缓存拉近数据距离, 并利用硬件缓存一致性实现多核间纳秒级的就地数据共享与免内存同步, 从而掩盖了单核计算、主存访问及多核协作的等待延迟.

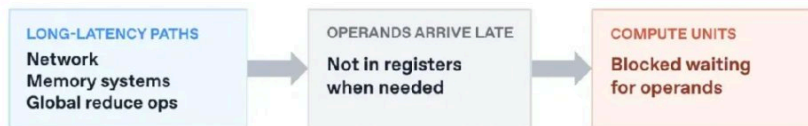
与之相对, GPU将绝大部分晶体管预算倾斜给密集的算术逻辑单元, 通过SIMT架构组织起数以万计的并发线程, 依托硬件级超快线程上下文切换在某些线程等待访存时瞬间调度其他就绪线程, 但是它根本不在乎单条指令的快慢, 而是用绝对的并行度将数据流的整体处理延迟彻底淹没在海量的计算任务中.

而对于 LLM 推理来看, 它刚好落在下表中 A\* 这个位置. batch=1 时一层就是几个 GEMV, 同时还要在毫秒量级穿过数十层, 还要在每层中间做完大量的并行计算后的结果同步.

|       | 延迟敏感 | 延迟不敏感 |
|-------|------|-------|
| 数据规模大 | A*   | GPU   |
| 数据规模小 | CPU  | NA    |

同时 OpenAI 在 HotChips 上的 Session 有一页专门阐述了这个问题: 我们实际测量到的延迟, 很大程度上是由底层架构本身决定的.

## Architecture can dominate observed latency



### What causes long latencies and late operands?

Unified memory subsystems tend to highly contended paths  
Independent cores out of sync make global memory fences expensive  
Centralized resources mediating network access

→ NOT hardware speed of light

### Utilization falls when compute/memory blocked waiting on data in transit

OpenAI

图中的 长延迟路径 → 操作数迟到 → 计算单元阻塞 为传统的 roofline model 补上了另一个维度, 对于 GPU 芯片而言并不是简单的 Compute bound 和 memory bound 的 workload 区分, 而是还要加上一个 阻塞, 此时没有任何资源被打满, 但所有资源都在等. 利用率掉下去, 而两条天花板都还很远.

### 1.3 Nvidia GPU 的缺陷

我们在这一节详细展开阐述一下 OAI 那一页的观点.

首先我们来谈 OAI 提到的 Unified memory subsystems tend to highly contended paths, 它采用统一 L2 + 全局编址, 任何核等价访问任何地址. 但是这样的统一访问虽然编程上隐藏了硬件的复杂性, 但是也带来了大量的工程约束. 例如 SM 太多带宽需求太大, 从 Ampere 开始 L2 拆成了 2 个 Partition, 在 Hopper 上 cross partition 会增加 200 cycles 的延迟. 而在 Blackwell / Rubin 上还会带来更多的影响, 每个 Die 有两个 L2 partition, Dual-Die 会有 4 个 Partition. 在 Blackwell 上会增加接近 400 Cycles 的延迟, Rubin 还需要考虑到 HBM4 的累计带宽已经快接近 L2 带宽了, 同时还要受到 NV HBI 的影响, 我个人认为 Unified Memory Access 将会导致 HBM 22TB/s 的实际带宽利用率只有 60%~70%. 并且当并发请求超出端口服务能力时, 产生的排队延迟 (Queuing Delay) 其开销远超数据传输本身.

另一个问题是在一些数据同步和 warp 调度上, 在现代的 GPU 架构中, 每个计算核有了独立 PC、自由异步推进, 并且配合 TMA 这些异步内存访问器件增加了吞吐, 但正因为计算核是独立的, 硬件就无法预知谁在什么时候到达, 于是一次全局 fence 必须真的去“问一圈”, 甚至在一些算子上需要整个 system level cross ScaleUP/ScaleOut network 进行 barrier. 这就是 OAI 谈到的 Independent cores out of sync make global memory fences expensive.

最后 Centralized resources mediating network access 讲的是当成百上千个核心需要向外部网络发起通信时, 若通过集中式的 DMA 控制器、MMIO 接口或共享网卡端口进行收发, 仲裁逻辑 (Arbiter) 会成为严重的吞吐瓶颈与延迟热点, 从而将硬件本身的极速传输能力彻底拖垮. 这个涉及到一系列复杂的互联系统的问题, NOC / ScaleUP / ScaleOut 都有各种问题, 例如 KVCACHE 拷贝时采用的 CopyEngine 或者 ScaleOut NIC 上 doorbell 的约束, 以及很多通知无法支持直接写到 SMEM, 因此只能写到一些 Remote GMEM, 然后 CUDA Core polling GMEM.

正是因为这些原因, 我过去几年才会对 Nvidia 提出 SM 内需要一个超标量核让编程者能够更好的控制指令发射和调度, 同时在 L2 上给用户更多的控制能力, 进一步和 SM 内的 SMEM 融合... 而恰好 OpenAI Jalapeño 也作出了这样的选择, 下一章我们将从推理业务第一性原理的角度逐渐展开, 如何设计一颗适合推理的芯片.



## 2. 如何设计推理芯片

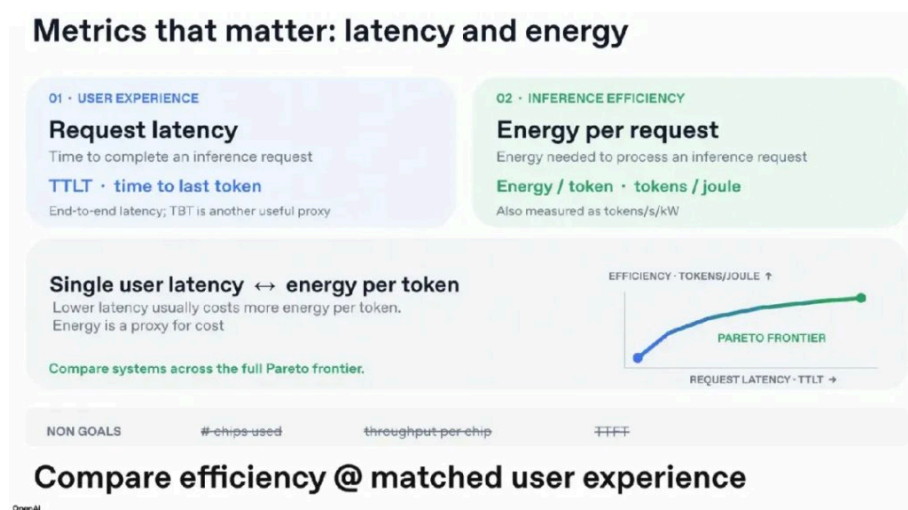
我们仔细分析发现前一节所讲的缺陷, 它们全都是通用性的代价, 需要强调, 这并没有在说 GPGPU 设计得差, 因为每一条决定当初都是对的, 它是 Nvidia 成功的关键, CUDA SIMT 的编程范式在过去十多年极大的简化了大家编写并行计算程序的复杂度. 但是这个前提在 LLM 推理场景带来了大量的问题, 然后伴随着 Coding Agent 的发展, 各种 Auto-Kernel(eg. KDA)百花齐放, 似乎我们也不需要为这些通用性付出高昂的代价了...

但这些仅仅是从芯片体系结构的视角, 更重要的是, 我们需要考虑从整个推理业务视角作为第一性原理.

### 2.1 推理芯片第一性原理

从第一性原理来看, 以一个推理服务经营的视角主要就是基础设施成本和供应链约束下, 满足用户体验的SLO下尽可能多的输出 tokens. 而供应链在北美最大的约束可能主要是在电力供应上.

因此 OpenAI 的演讲稿故意不把chips used,throughput per chip和TTFT列为主目标. 它选择的是 request-level user experience 和 energy per request.



吞吐能够通过并发掩盖某些等待, 单个任务的依赖链却不能凭空并行. 因而使用TTLT, 即time to last token, 表示整个请求完成时间, 同时用TBT, 即 time between tokens, 近似流式交互速度. 但我们注意到, 降低 batch 通常会改善TBT, 却让权重和网络复用下降, 每token能耗上升. 提高 batch 则通常提高tokens/s/kW, 但请求排队和每用户token间隔变长. 因此存在一个响应延迟 (TTLT) vs 能效 (tokens/Joule) 的 Pareto 前沿. 形式化的描述为:

$$\text{maximize} \quad \frac{\text{requests}}{\text{second} \cdot \text{watt}}, \quad \text{s.t.} \quad t_{\text{E2E}} \leq \text{SLA}$$

而 LLM 推理, 实际上是 CPU 界研究了三十年的单线程延迟问题...



| CPU 界的答案             | LLM 推理里的对应物                            |
|----------------------|----------------------------------------|
| 分支预测: 猜下一步, 错了回滚     | 投机解码: 猜后 $\gamma$ 个 token, 错了丢弃        |
| 乱序执行: 在依赖链的空隙里找独立工作  | 乱序核                                    |
| 缓存 + 预取: 把延迟从关键路径上挪走 | 硬件管理 L1 + 依赖良好的预取, 而非 scratchpad + DMA |
| SMT: 用另一个线程填充空隙      | 用另一条序列填充空隙 (请求级批处理)                    |
| 加宽发射宽度收益递减 (ILP 上限)  | 加投机深度收益递减 (接受率上限)                      |

那么从体系结构的视角来看得出一个很简单的结论:

尽量提高 Data Locality, 降低数据搬运的成本.

从E2E延迟的视角, 进一步提高指令执行的并行性, 降低空闲等待的时间.

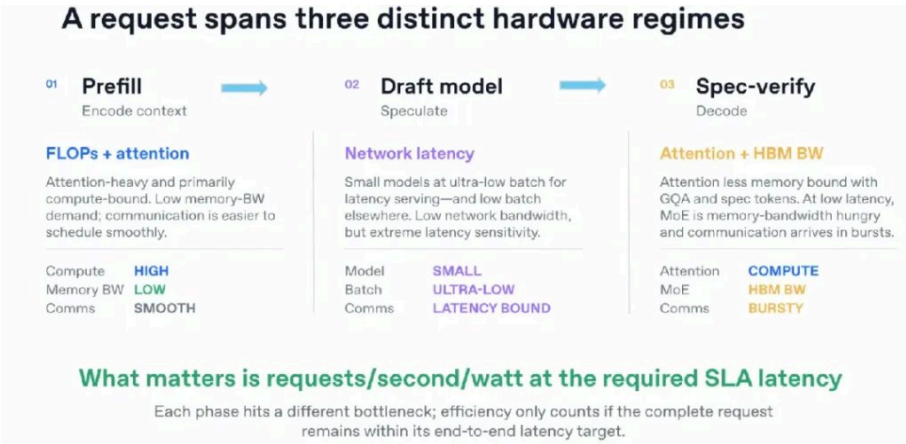
接下来几个小节我们将从推理的技术细节详细展开分析...

## 2.2 推理 Infra 现状

### 2.2.1 不同阶段的性能需求

通常对于一个推理系统来看, 主要业务分为Prefill, Decode, SpecDecode 三块, 可能我们原来的定性的结论是 Prefill是 Compute Bound, Decode是 Memory Bound, 但实际上我们从E2E 延迟的视角还需要考虑 SpecDecode 时的 Draft 模型和 Verify 两个阶段, 它们都会处在roofline 的不同位置.

OpenAI在 hotchips 的session 上也谈到这个问题, 但是其表述是不完善的. 例如在第三格, 主标题是Spec-verify, 副标题是Decode,但的自回归decode与的批量verify处在roofline上两个不同位置, 相差倍算术强度.



因此我们严格的把它分为四块, 为了简化所分析的问题, 此时我们以单个 Request 的处理进行分析

**1. Prefill:** Prefill一次处理完整输入上下文. 对 hidden-dim 为, 长度为 的普通Transformer模型, projection 与 FFN 的主计算量近似随 增长, full attention 的 score 部分随 增长. 大 带来足够的 M 维度, 权重可以在许多 token 之间复用, 所以 tensor engine 容易进入高利用率区. 因此它是一个 Compute-bound 的计算, 即 compute high, memory BW low, 另外对于通信而言, 它的 collective 消息随 增长, 属于带宽受限而非延迟受限, 因而容易平滑调度.

**2. Decode:** 自回归模型在 Decode 阶段每个序列每次只处理 1 个新 token, 即, 矩阵计算退化为 GEMV, 这意味着一份权重只服务一行, 在短或中等 context 下通常受 Weight HBM 带宽限制, 在长 context 下会逐渐转为 KVCache HBM 带宽限制. 在分布式系统中, 与此同时每层边界仍要做一次collective, 消息只有大小, 小消息 collective 的网络同步延迟可能成为第一瓶颈.

**3. Draft Model:** 通常和模型相关. 经典自回归草稿模型和EAGLE类方法需要执行多次串行起草步, 因而经常受权重/KVCache带宽和同步延迟限制. DFlash使用一次块并行扩散主干网络前向传播同时生成整个草稿块. DSpark使用一次并行主干网络前向传播, 再叠加一个低成本Markov或RNN串行头. 对DFlash/DSpark而言, 草稿权重只需按主干网络调用读取, 算术强度可随 提升, 瓶颈可能转向矩阵算力, 目标隐藏特征/KV注入带宽, vocabulary projection 或轻量串行采样延迟.

**4. Verify:** 验证的物理token数也不一定等于固定的. 线性草稿通常验证一个前缀, 树形草稿需要验证全部树形节点, DSpark则按置信度和实时硬件容量为每个请求选择不同的验证长度. 稠密权重可以跨本轮物理验证token复用, 但是MoE目标模型可能因为去重后的专家数量随token预算增长而失去大部分权重复用. 高效验证注意力必须让一个前缀KV分块被同一请求的全部候选查询或树形节点复用.

总结成一个表如下:

Pressure matrix of the four inference phases on chip resources

9 phase and algorithm subtypes × 5 resource dimensions. Filled blocks give the pressure level and frames mark limits. A rating is relative pressure on chip design and phase performance, not an absolute traffic figure

| Phase and algorithm subtype |                             | Compute                                                                                                                                                  |                                                                                                                                                                   | Memory                                                                                                                                                     |                                                                                                                                                            | Communication                                                                                                                                     |                                                                                                                                                                        |
|-----------------------------|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                             |                             | Matrix compute                                                                                                                                           | HBM weight BW                                                                                                                                                     | HBM KVCache / target-feature BW                                                                                                                            | Network BW                                                                                                                                                 | Network sync latency                                                                                                                              |                                                                                                                                                                        |
| Prefill                     | where prompt in one pass    | <div><div></div><div></div><div></div><div></div><div></div></div> 5/5<br><div>1M - 2L large GEMM, plus O(N<sup>2</sup>) attention at long context</div> | <div><div></div><div></div><div></div><div></div><div></div></div> 3/5<br><div>weights streamed over full tokens, short prompt reuse if</div>                     | <div><div></div><div></div><div></div><div></div><div></div></div> 3/5<br><div>mostly near-KV reads from a cold cache, peak for who reads</div>            | <div><div></div><div></div><div></div><div></div><div></div></div> 4/5<br><div>TP + dist. EP + dist. all, and CP also moves KV blocks</div>                | <div><div></div><div></div><div></div><div></div><div></div></div> 3/5<br><div>many collections, but large messages and fine overlap</div>        |                                                                                                                                                                        |
|                             | Decode                      | one token per step                                                                                                                                       | <div><div></div><div></div><div></div><div></div><div></div></div> 3/5<br><div>low overhead FLOPs, GEMV and an all-N efficiency in what matters</div>             | <div><div></div><div></div><div></div><div></div><div></div></div> 3/5<br><div>every token scans the active weights; MoE reuse is critical</div>           | <div><div></div><div></div><div></div><div></div><div></div></div> 4/5<br><div>reads the whole history KV per step, traffic scaled</div>                   | <div><div></div><div></div><div></div><div></div><div></div></div> 2/5<br><div>small messages, higher EP and CP degree raises it</div>            | <div><div></div><div></div><div></div><div></div><div></div></div> 3/5<br><div>small per-layer collections, K/bytes, repeated per token</div>                          |
| Draft                       | autoregressive / EAGLE      | up to full drafting steps                                                                                                                                | <div><div></div><div></div><div></div><div></div><div></div></div> 3/5<br><div>small-mode GEMV or small-M GEMM, low-cache efficiency matters</div>                | <div><div></div><div></div><div></div><div></div><div></div></div> 4/5<br><div>weights re-read up to y times when not re-used</div>                        | <div><div></div><div></div><div></div><div></div><div></div></div> 3/5<br><div>draft KV reads as y, rising to 4/5 at long context</div>                    | <div><div></div><div></div><div></div><div></div><div></div></div> 2/5<br><div>small when co-located: cross-node TP/GP raises it</div>            | <div><div></div><div></div><div></div><div></div><div></div></div> 3/5<br><div>full backbones, classes, and sampling from a select serial path</div>                   |
|                             | DFlash block-parallel       | one parallel backbone pass                                                                                                                               | <div><div></div><div></div><div></div><div></div><div></div></div> 4/5<br><div>N × D by small M GEMM, block attention, shared LM head</div>                       | <div><div></div><div></div><div></div><div></div><div></div></div> 3/5<br><div>backbone read once for KV, however, small KV still accesses 4/5</div>       | <div><div></div><div></div><div></div><div></div><div></div></div> 4/5<br><div>multi-layer target features, KV injection, long draft reuse</div>           | <div><div></div><div></div><div></div><div></div><div></div></div> 3/5<br><div>small when co-located: cross-node TP/GP raises it</div>            | <div><div></div><div></div><div></div><div></div><div></div></div> 2/5<br><div>no y full serial, block-sampled still reuses one sub-boundary</div>                     |
|                             | DSpark semi-autoregressive  | one backbone + light serial heads                                                                                                                        | <div><div></div><div></div><div></div><div></div><div></div></div> 4/5<br><div>parallel backbones plus y low-rank-to-head bias projections</div>                  | <div><div></div><div></div><div></div><div></div><div></div></div> 3/5<br><div>backbone read once, Markov/RNN head weights should stay re-used</div>       | <div><div></div><div></div><div></div><div></div><div></div></div> 4/5<br><div>target features and KV injection as DFlash, plus confidence state</div>     | <div><div></div><div></div><div></div><div></div><div></div></div> 3/5<br><div>the parallel backbone sends large messages; keep heads local</div> | <div><div></div><div></div><div></div><div></div><div></div></div> 3/5<br><div>only y light heads and sampling remain, not full backbones</div>                        |
|                             | tree / DDTree / JTree       | depth y, node budget Q <sub>y</sub>                                                                                                                      | <div><div></div><div></div><div></div><div></div><div></div></div> 3/5<br><div>top-K, tree building, structured attention, per-node flags</div>                   | <div><div></div><div></div><div></div><div></div><div></div></div> 3/5<br><div>depends on the base draft model: the output head may dominate</div>         | <div><div></div><div></div><div></div><div></div><div></div></div> 3/5<br><div>tree metadata and draft cache, target serial blocks under verify</div>      | <div><div></div><div></div><div></div><div></div><div></div></div> 3/5<br><div>depends on the base draft model and the node budget</div>          | <div><div></div><div></div><div></div><div></div><div></div></div> 3/5<br><div>confirm path existence, CPU heads, mask compilation or GPU/CPU steps add barriers</div> |
| Verify                      | fixed-length linear prefix  | linear budget Q <sub>y</sub> = Q <sub>0</sub> × (1 + r) <sup>y</sup>                                                                                     | <div><div></div><div></div><div></div><div></div><div></div></div> 4/5<br><div>N to Q<sub>y</sub> target GEMM, one acceptance needed compute</div>                | <div><div></div><div></div><div></div><div></div><div></div></div> 4/5<br><div>long profiles that, needs as y + 1 repeated for block reuse</div>           | <div><div></div><div></div><div></div><div></div><div></div></div> 4/5<br><div>multi-accept, which one found the other all</div>                           | <div><div></div><div></div><div></div><div></div><div></div></div> 3/5<br><div>only one group of target sub-tasks, yet every layer steps</div>    |                                                                                                                                                                        |
|                             | tree                        | weight Q <sub>y</sub> = Σ <sub>i</sub> w <sub>i</sub>                                                                                                    | <div><div></div><div></div><div></div><div></div><div></div></div> 3/5<br><div>every node runs target projections, MoE and the output head</div>                  | <div><div></div><div></div><div></div><div></div><div></div></div> 4/5<br><div>distinct accepts track physical nodes, not accepted depth</div>             | <div><div></div><div></div><div></div><div></div><div></div></div> 3/5<br><div>all nodes with serial KV only the accepted path survives</div>              | <div><div></div><div></div><div></div><div></div><div></div></div> 3/5<br><div>TP/GP traffic scales with all nodes</div>                          |                                                                                                                                                                        |
|                             | DSpark confidence-scheduled | dynamic Q <sub>y</sub> = Σ <sub>i</sub> (1 + r) <sup>i</sup>                                                                                             | <div><div></div><div></div><div></div><div></div><div></div></div> 4/5<br><div>delta all operations point, marking Q<sub>y</sub> under read drops it to 3/5</div> | <div><div></div><div></div><div></div><div></div><div></div></div> 4/5<br><div>delta all accepts and weight traffic follow the dynamic Q<sub>y</sub></div> | <div><div></div><div></div><div></div><div></div><div></div></div> 4/5<br><div>peak KV and operations scales follow Q<sub>y</sub>, needs wider reuse</div> | <div><div></div><div></div><div></div><div></div><div></div></div> 4/5<br><div>TP/GP traffic scales with all nodes</div>                          | <div><div></div><div></div><div></div><div></div><div></div></div> 3/5<br><div>async planning: Note the local, sync CPU scheduling is a barrier</div>                  |

**Pressure level: number of filled blocks**

|  |                                                       |
|--|-------------------------------------------------------|
|  | very low, rarely a design constraint                  |
|  | low, mostly existing in some configs                  |
|  | medium, not having a secondary bottleneck             |
|  | high, often near the bottleneck                       |
|  | very high, usually the limiter or a key design target |

**Cell markers**

|  |                                               |
|--|-----------------------------------------------|
|  | this resource directly caps the phase         |
|  | Read with corresponding                       |
|  | limited control or scheduling, not the design |

**Resource groups**

|  |                             |
|--|-----------------------------|
|  | Compute, 1 dimension        |
|  | Memory, 2 dimensions        |
|  | Communication, 2 dimensions |

**Variable legend**

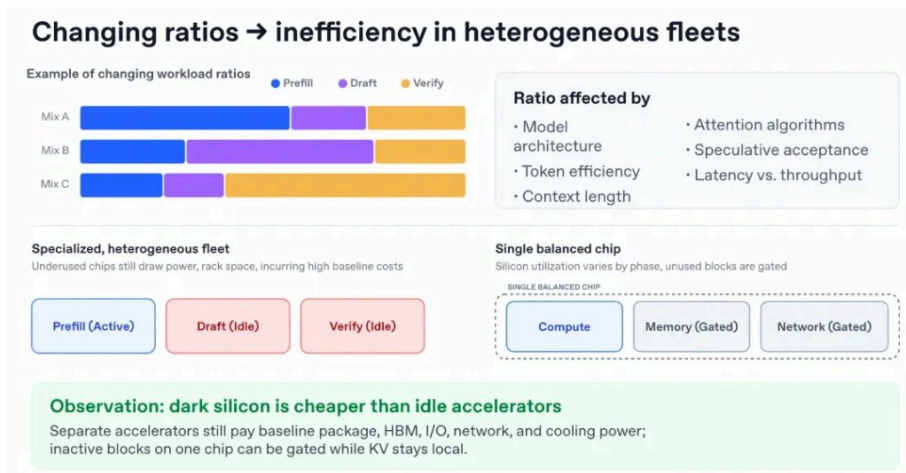
|           |                                                                                         |
|-----------|-----------------------------------------------------------------------------------------|
| $R$       | extended requests or operations in the current batch                                    |
| $E$       | total prompt length in the draft phase                                                  |
| $S$       | prefix lengths already present before this model call                                   |
| $N$       | number of backbones / steps                                                             |
| $d$       | model hidden dimension                                                                  |
| $A$       | KV for every head: $A_{H,K}$ , single-size K or $w$ in width is $d$ per token per layer |
| $E$       | total requests per MoE layer, with the token count $R$ for the distributed expert count |
| $r$       | experts each token is routed to, the MoE top- $r$ parameter                             |
| $y$       | max draft length of one speculative step: usually the max depth for tree drafts         |
| $R$       | active requests in the current verification batch                                       |
| $n$       | request index used in $\Sigma$ attributed to the MoE top- $r$ parameter $r$             |
| $E$       | draft prefix length verified per request in fixed-length linear verify                  |
| $Q_y$     | draft prefix length actually verified: the request $w, n, y, Q_y, Q_0$                  |
| $n_y$     | physical tree nodes to verify for request $y$                                           |
| $Q_0$     | total physical tokens or nodes processed in one target verification                     |
| $M$       | effective node or token dimension of the current matrix op, not model capacity          |
| $p_n$     | prefix KV read amplification of request $w, n, y$ : means one read per prefix token     |
| $Q_{H,K}$ | order of long-context attention cost, not a scalar variable                             |

Default settings: medium-to-long context, low-to-medium batch, target model on distributed TP/GP draft and verify on localized where possible.

Draft and verify are expanded by algorithm subtype but still falling to the four logical inference phases. Ratings shift with TP, D, GP, E, r, y, Q<sub>y</sub>, precision, the acceptance curve and the parallel strategy. Composite terms are products of legend variables: R, prefix tokens, E, S, and dist. TP and GP traffic, R, target KV reads, R, draft positions, y, draft KV reads, Q<sub>y</sub>, and verify GP traffic.

可以看到在不同阶段对于芯片的计算, 内存, 网络带宽, 网络同步延迟 的需求会存在巨大的不同. 如果我们采用专用的芯片应对每个阶段, 对于资源的配比也会出现问题. OpenAI 也谈了这个问题:





OAI阐述的是当前各种 PD 分离和 AFD/SpecDecode 这样的 LPU 部署带来的缺陷. 原页面沿用它自己的三分法, 所以只画了prefill, draft, verify三个比例. 实际上按照我们前面所分的4个阶段分别放入4种专用设备, 看似能让每种芯片都针对自己的kernel做到最优. 问题在于设备台数在部署后是离散且相对固定的. 设4个池的能力为, 到达工作量为, 下标依次对应prefill, decode, speculate与verify, 则可服务速率受最紧张的那个池限制:

例如我们来分析 SpecDecode 接受率动态变化的影响, 与此消彼长, 而拨动它们的不是请求. 一个请求的输出token只有两条出路, 走纯decode, 或者走 Draft + Verify. 记输出token中经 SpecDecode 路径提交的比例为, 为每秒已提交的输出token数, 则

其中是一轮 Draft Model 的成本, 自回归为步而块并行为1次宽pass, 则正比于该轮的物理 token 预算. 不由请求混合决定: 塌陷把它压向0, 高并发下调度器按前缀存活概率裁掉低置信后缀也把它压向0, 而接受率回升时它又回到1.

于是在同一批请求, 同一个模型, 同一个context length下, decode池与verify池的负载可以走向两个相反的极端. **decode池必须按配齐, draft与verify池必须按配齐, 而这两个峰值不会同时到来:** 资源预留的是峰值之和, 用的是其中一组.

四个阶段的限制项落在不同资源轴上: prefill是矩阵算力, decode是HBM权重带宽加逐层同步延迟, 自回归起草是draft权重带宽加次同步, verify是去重expert的权重流量加EP all-to-all突发. 所以这四种专用芯片不是同一款芯片的大小号, 而是算力, 带宽与网络配比互不相同的四种芯片. 接受率上升时, verify的次数下降; context length上升时, prefill与KV的成本提高; 模型换成expert占比高的MoE时, verify的网络与HBM压力又增加. 一个pool成为瓶颈时, 另外两个pool仍要支付 package, HBM, I/O, 网络与散热的基线成本. 由此得到一个容易漏掉的后果: 即使运维愿意把设备从闲置的池搬进 瓶颈池, 搬过去也无法使用, 因为prefill芯片的HBM与算力之比本来就不是按 decode配的. **固定的设备台数比例等于同时锁死五个资源维度的比例.**

## 2.2.2 KVCache 的代价

在传统的推理部署中, 我们通常会考虑 PD 分离的部署方式, 而此时大量的 KVCache 的搬运也带来了极大的功耗以及网络同步的损失. OpenAI 在 hotchips 中的阐述如下:

这一页把架构选择压成一个二选一: 异构要么发生在系统之间, 代价是 KV 跟着不同的阶段走, 这就是现在 PD 分离的做法, 但是每个阶段的网络和同步所带来的影响以及KVCache 搬运的功耗是巨大的.另一种选择是将不同阶段的 KVCache 移动放在芯片之内, 代价是活跃资源的比例跟着不同的阶段变化.

通常 KVCache 的大小为 和序列长度成正比, 即便是一些 Linear Attention 的算法也需要在多次交互中为每一轮留下 Snapshot. 数据的搬运会导致 ScaleUP/ScaleOut 以及 NOC 对原有的集合通信的性能影响, 同时还要考虑大量的 KVCache 从 Prefill 节点的 HBM 读出, 再从 Decode 节点 HBM写入, 依然会影响到原有的推理算子的执行. 具体的分析可以从时间比, 每轮边界, HBM容量与事务语义 四个方向展开进行量化的分析.

由于涉密这部分就不公开多说了...还记得本文开头所讲的我们 MaaS 服务将传输协议换成 CIPU eRDMA 后, 推理引擎未做任何代码修改 TTFT 下降了 40%...

最后对于芯片的设计, 如果用几种独立加速器, KV cache 需要在设备之间移动. Jalapeño 的策略是把异构能力放在一颗平衡芯片内部:

当前阶段需要 tensor compute, 就激活更多计算单元.

当前阶段受 HBM 限制, 就强调本地内存路径.

当前阶段通信密集, 就使用 collective 和 scale-up 网络.

暂时不需要的模块进行 power gating.

KV cache 尽可能留在本地.

### 2.3 推理芯片体系结构

几个月前 David Patterson 有一篇文章《Challenges and Research Directions for Large Language Model Inference Hardware》<sup>[1]</sup> 在阐述推理硬件的体系结构, 对于内存子系统, 使用 HBM / 3D-DRAM / HBF 有很多讨论, 但是为了简化问题, 例如新的 HBM 可以构建 PNM 和 Customized Based Die 将 Memory Controller 和 HBM PHY 从计算 Die 中移出获得更多的算力资源, 或者像 AMD 那样使用 XCD 和 FCD 的 3D 封装, 可以进一步提高 XCD 的良率以及更大的 L2 Cache 和互联NOC, 我们暂时不讨论这部分的内容, 更多的关注在计算 Die 内部的微架构以及互联架构.

具体的内容在公司内部也有一个详细的文档分析了各种算力芯片的微架构, 包括 Nvidia GPGPU / LPU / Google TPU 和华为 Ascend 950. 并且针对 Agent based compiler 这些工作进行了一系列探索.

#### 2.3.1 编程接口的变化

前几个月我在做一个 Anthropic 公司招聘用的性能工程相关的家庭作业面试题, 任务代码在: [https://github.com/anthropics/original\\_performance\\_takehome](https://github.com/anthropics/original_performance_takehome), 大概的内容是在一个自定义的 VLIW + SIMD 处理器上进行性能优化. 通过使用Claude-Opus模型进行优化, 后来刷到第二以后因为差距已经在2%以内, 以及一些计算资源和 token 费用的问题就没有继续做下去了, 仿佛现在还是第三...

通过这个题目让我意识到它就是在设计一个专用 ASIC 芯片用于推理和训练的编译系统, 似乎原有的 CUDA 生态和 SIMT 的编程接口虽然对人友好, 但带来的资源开销是巨大的, 然而有了 Agent as compiler 了以后, 并且伴随着 Ligeng's KDA 这样的工作, Kernel 优化似乎可以通过 Agent 很好的完成了, 似乎再也不需要去照顾人类的心智缺陷, 而充分的发挥硬件的能力.



那么原有的 SIMT 的抽象和 GPGPU 在推理场景下的优势也就不存在了, 通常一个模型的架构在预训练阶段就已经定死, 我们会有两个月的时间充分的对推理平台进行基于 Agent 的调优. 这也是 OpenAI Jalapeño 体系结构上解除的一道枷锁.

### 2.3.2 体系结构的取舍

去年的blog也讨论过一个问题, Nvidia 的 Warp 调度机制和延迟隐藏机制并不适合推理场景. 另一方面在文章《谈谈GPU的内存模型及互联网络设计》中也谈到了 Nvidia GPGPU 复杂的内存语义, 数据路径上存在 Genric Proxy 和 Async Proxy, 如果需要 ScaleOut 还要额外的同步机制, 然后整个 NOC 和内存层次结构也越来越重, 而 Warp 调度的机制来看并不能让我满意, 每一次数据搬运都需要显式编排, 而编排本身有开销, DMA 描述符的构造、启动、以及等待完成的 barrier 等等...

假设一个 tile 的计算只需要 500 个周期, 而 DMA 编排 + barrier 开销是 100 个周期, 那么固定开销占了 20%. 但是在 GPU/TPU 的典型场景 (大 batch, 一个 tile 算几千周期) 里, 同样的 100 周期只占 2%, 完全可以忽略. 很不幸的是, Decode 阶段正好落在前一种情况: batch 小, 每个 tile 的计算量小, 于是同步开销从“可忽略”变成“主导项”. 这时候把同步从“显式 barrier”换成“硬件依赖跟踪 (乱序 + cache)”就有了正收益.

因此, 我希望他们能在 SM 内加一个超标量核进一步来控制延迟, 正好这次 OpenAI Jalapeño 中的 Out-of-Order Core 正好也验证了这个判断.

同时, 前一段时间也对 Blackwell 系列进行了完整的指令 Cycle 级的测量和完整的芯片级分析, 对外公开了《Dissecting SM\_120 through Microbenchmarking》<sup>[2]</sup>. 而对于 B300 片上网络 / L2 Cache 和功耗管理上也发现了大量的问题. 简而言之就是整个内存层次结构太重了, 而片上网络中, 为了维持 L2 的带宽, 很早就被设计成了 2 个 Partition, Nvidia 期望继续维持 UMA 的编程模型来隐藏不同 L2 Partition 访问的延迟差, 但是在 Multi-Die 的 Blackwell / Rubin 上代价太大, MultiDie的结构使得它单颗 GPU 有了 4 个 L2 Partition, 再维持 UMA 在推理阶段引入的访问内存损失就多达几百个周期, 显然是得不偿失的.



另一方面是在 RDMA 这些场景, 很多通知机制我希望能够 bypass L2 直接写到 SMEM, 这样 SM 就可以很快速的 polling SMEM 获得相对确定性的延迟, 而当前 polling GMEM 很有可能会被 warp scheduler yield导致更大的延迟. 并且 L2 Cache 的机制我们是很难控制的, 很有可能我们关键的一些同步用的 counter 被 SM 大量的内存访问 evict, 我一直也在跟 Nvidia 提什么时候能够把 L2 Cache 也划分出一部分作为 SMEM 那样可以自己控制的器件....

还有进一步的一些问题, 例如由于同步机制的影响, 无论是 intra-chip 还是 inter-chip 的集合通信性能都会受到延迟的影响, 这也导致了场景下 TP 并行的规模无法进一步扩展成为显著的影响点. 而这些问题似乎都来自 UMA 的影响, 那么很自然的一个想法, 能否做成一个 NUMA 结构, 并将关键的集合通信相关的处理在 NOC 和互联(ScaleUP/Scale Out)进行充分的优化?



当然我能理解 Nvidia 作为一个商业公司要去追求 peak throughput 的原因, 但是对于推理系统而言, 我们从目标函数出发, 有两条独立的推论, 分别落在微架构和系统架构上.

**推论 A:** 有效 token 必须靠“低延迟下的高利用率”取得, 不能靠堆 batch, 具体来看:

- 缩短访问内存的路径: 对于内存延迟放弃隐藏的策略, 而通过更少的内存层次消除, 可以接受 NUMA 的代价
- 更高的单指令流的ILP: 使用乱序核 + 硬件 L1 + 预取的方式, 而不依赖 occupancy
- 适当的矩阵单元大小: 保证小batch时也不会有性能损失
- 消除 Kernel Launch 与同步的固定开销

**推论 B:** 芯片面积不再是稀缺资源, 闲置整机才是浪费.

- 拒绝异构的各种分离技术, 同构集群避免专用资源调度瓶颈
- 不以 peak FLOPS 为目标, 充分考虑推理各阶段的资源需求, 做到算力/内存带宽/通信的平衡
- 允许部分的暗硅, 保证芯片满足各阶段的需求
- 平衡整个芯片的功耗, 并以它作为芯片设计目标
- 降低通信的延迟, 特别是长尾延迟

整个在体系结构上的决策可以概括为下面这张图:

而这些体系结构方面的思考和 OpenAI 不谋而合, 具体的分析我们将在下一章展开.

### 3. OpenAI Jalapeño 芯片架构

Jalapeño 应该是工业界第一款 AI 加速的软硬件协同设计的芯片平台, 硬件上从第一行 RTL 到最终 tapeout 仅用了 9 个月, 当然这和整个芯片架构也紧密相关, 由于没有复杂的同步和硬件调度机制, 这使得验证的周期加快了很多.

另外为了time-to-market, 整个芯片复用相对成熟的Ethernet based Scale-UP. 整个工业界第一个基于标准 Ethernet 构建的支持内存语义的 ScaleUP 应该是我在 2020 年开发的 NetDAM, 这也是一个很简单的工程上的 trade-off, 通用交换机芯片的供应链非常好, 成本也很低, 虽然延迟和有效利用率还有一些缺陷. 但是对于推理芯片互联所需要的极致低延迟互联, 这只是一个暂时的方案, 即便是后面 BRCM 推出的 SUE 也是较大的缺陷的, 跟他们交流的过程中, 我告诉他们可以参考 FinePack 的一些方案, 但是最后发现 BRCM 并没有完整的知识背景, 没有考虑到一些非常细节的排队延迟, 特别是一些长尾延迟的抑制...

#### 3.1 芯片架构 Overview

Jalapeño 的芯片架构如下图所示:

它放弃了 Nvidia 那样的 UMA 结构, 而是将计算核和 HBM Slice 一对一的进行配对, 构建最低延迟的内存访问路径. 然后片上网络专门针对cross-die 集合通信进行了特殊的低延迟大带宽优化. 而且还保留了一条用于兜底的基于通用以太网的 ScaleUP 互联路径.

这样的设计也是软硬件协同的结果, Jalapeño 的很多硬件简化 (删掉统一 L2、放弃跨卡内存语义、暴露 NUMA 距离) 都把复杂度推给了编译器和 kernel. 我们注意到底部的那句话, 实质上就是通过**explicit placement**, 数据落在哪个 slice 是**被明确安排的**, 并通过编译器构建了很好的优化, 后面我们将在软件系统架构中详细展开. 同时, 芯片没有中央调度器, 每个 core 独立取指与推进, 这样的好处是防止"数据移动和同步"主导执行时间.

整颗芯片采用多个 Die 封装, 如下图所示:

Compute Die 采用 TSMC N3P 工艺, 基本上顶满了 reticle-size, 整体算力 MXFP4 支持到 13.4 PFLOPS, 外接支持 6 颗 HBM4. 但是将 IO 独立到一个单独的 IO-Die, 并采用 TSMC N3E 的工艺. IO 与计算解耦, 意味着下一代可以只换 compute die 而复用 IO chiplet, 或者反过来, 同时也降低了对 Compute Die 的资源占用. 对一个刚起步、要快速迭代的项目, 这是有价值的自由度. 并且整个 ScaleUP 并没有极致的追求更大的带宽, 仅支持 600GB/s local(in-rack) 和 200GB/s global(cross-rack).

另外我们注意到它的 Matrix 引擎原生支持 MXFP8 × MXFP4, 这是 MoE 中常用的激活 FP8 × 权重 FP4 路径, 另外整个芯片应该还是有针对 Attention 计算的 BF16 支持能力.

### 3.1.1 Core Slice 架构

从已知的公开信息, 我们**推测**它的计算核结构如下:

首先整个程序采用一个 GigaKernel 执行, 类似于 CUDA 中的 Persistent Kernel, 但是由于 Jalapeño 更多的要考虑延迟, 因此不会有 CUDA Core 那样的多个 warps 占用的情况, 更多的是靠内部的超标量结构构建的单个指令流内部的预取与乱序窗口来隐藏延迟, 这就是它的前端取指 / 译码 / 乱序发射 的来源. 为了保证整个核心高性能的运行, 应当存在一个相对独立的同步锚点, 执行类似于GPU中的\_\_syncthreads / mbarrier 相关的能力.

然后在计算单元, 一般都是标准的 Tensor/Matrix Unit, SIMD/Vector Unit 和 Scalar Unit 构成. 最关键的部分在于 Matrix Unit 的架构. 我们期望它能够在推理 Decode 时小 batch 时有更高的利用率, 同时整体来看, 也能满足 Prefill 这样 Compute bound 的计算. 而这里便是整个芯片最关键的一环, 它是如何配合乱序执行? 最佳的Shape是多少? 延迟如何隐藏的? 我们在稍后的一节详细展开.

接着是内存子系统, 它并没有像 Nvidia 的 SM 那样配置 RegisterFile / L1Cache / SMEM / TMEM, 而是仅有一个 L1 Cache, 同时整颗芯片也没有一个全局的 L2 Cache. 然后 HBM 直接分 Slice 接到每个 Core Slice, 这样很好的避免了在 NOC 上的 contention.

和 Nvidia 最大的区别在于, 它的 Tensor Unit 和 SIMD Unit / Scalar Unit 之间不需要将数据频繁的在 SMEM / TMEM 和 Register File 以及 L1 Cache 之间频繁的进行搬运和拷贝, Tensor Unit 和 SIMD Unit 直接通过 L1 Cache共享操作数, Tensor Unit完成计算后, 直接就可以 issue SIMD 指令继续计算. 这样消除了 barrier / DMA编排等固定的开销, 然后 小的 Shape 进行计算对于 Decode 阶段的利用率也非常友好, 同时这块 L1 也可以很好的吸收掉 HBM 带来的抖动.

### 3.1.2 NOC 架构

在 NOC 上, 由于 Jalapeño 是一个 NUMA 架构, 并且在推理场景中大量的并行操作需要进行集合通信, 因此 OAI 为它构建了一个超低延迟的支持集合通信的独立 NOC. 而同时它还支持一个相对低速的传统的 NOC 用于 NUMA 节点之间的 HBM 内存访问和连接 ScaleUP IO Die.

### 3.2 Core Slice架构详解

我们需要考虑设计的这块芯片在Prefill, Decode, Draft和verify四个阶段都能够平衡的满足业务需求而不出现明显的短板, 并且在性瓦比(Perf-Watt Ratio)上获得数倍的收益. 那么针对 Decode batch小的情况, 需要详细考虑 Tensor Unit 的设计. 从第一性原理出发, 我们可以评估一些常用的模型的矩阵形状, 最终可以给出一个结论, 满足最小整除并不需要额外 padding 的 Tensor Unit shape 为 64x64. 然后考虑到整个系统是 latency-driven 设计的, 并且已知有大量的 Out-of-Order 的指令 issue. 并且整颗芯片移出了大量的 L2 Cache占用, Global NOC的资源占用, 因此我们可以在单个 Core Slice 内构建多个 Tensor Unit, 把芯片面积更多的花在 Tensor Unit 上.

因此我们猜测它的Tensor Unit是一个 64x64 @ MXFP4 的 Output-Stationary 的结构, 并且由于延迟的需求, 没有使用脉动阵列而是使用了一个常见的加法树结构. 然后整个 Core Slice 根据 13.4PFLOP/s 以及芯片频率 1.7GHz 倒推, 单个 Tensor Core MAC 为 64x64x2, 累计 64 个 Core Slice.  $13.4\text{PFLOPs} / 1.7\text{GHz} / 64 / (64 \times 64 \times 2)$ , 单个 Core Slice 包含了 15 个 64x64 Tensor Unit. 由于它是一个很大的 reticle size Die 的结构, 考虑到制造良率的影响, 而且整个芯片来看没有 L2 因此我可能会考虑放置 16 个 Slice 但仅激活 15 个的做法. 然后这些数据和公开的 MXFP8 等浮点性能也可以完全对齐. 稍微补充一下, 官方并没有公布 BF16 的峰值浮点性能, 我们可以根据这个结构倒推出来峰值在 835TFLOP/s. 可以看到相对于 B300 的 BF16 2.5PFLOPs 小了很多, 但这样的结构设计的取舍是值得的, 通常在推理过程中 Attention 的计算会用到 BF16 的精度, 而 Expert-FFN 的计算通常为 FP8xFP4, 而且 Attention 的计算中还要涉及到大量的 Softmax 等 SFU 处理, 通常每个 head 的 dim 也相对较小.

整个 Core 内的结构如下:



所有的操作都是只要有空闲的 Tensor Unit, Out-of-Order Core 可以进行多次提交提高指令并行度, 而数据依赖都由 L1 Cache 状态决定, 数据也可以硬件控制做Prefetch. 对于 L1 Cache 的大小, 我们根据 HBM 的带宽和延迟, NOC 互联的带宽以及 Tensor Unit 的处理能力等几个维度进行了详细评估, 每个 Core Slice估计 L1 Cache大小为 512KB. 对比 Blackwell 芯片, 实际上 Blackwell 芯片会有 RegisterFile / L1 Cache / SMEM / TMEM 等几块存储区域, 而 Jalapeño 很干净的处理为一整块区域并连接到 HBM Slice.

这正是整个 Jalapeño 的核心决策. 用更多的芯片面积给多个 Tensor Unit, 剔除复杂的 Global L2 / 调度器 以及 SM 内部的多种存储结构. 然后也无需追求更高的峰值 FLOPs 而是更加关注实际



工作中的 可维持的FLOPs. 然后在一些 workload 较轻时, 还可以通过 power gating 关闭掉一些执行单元, 获得更高的 Perf-Watt Ratio.

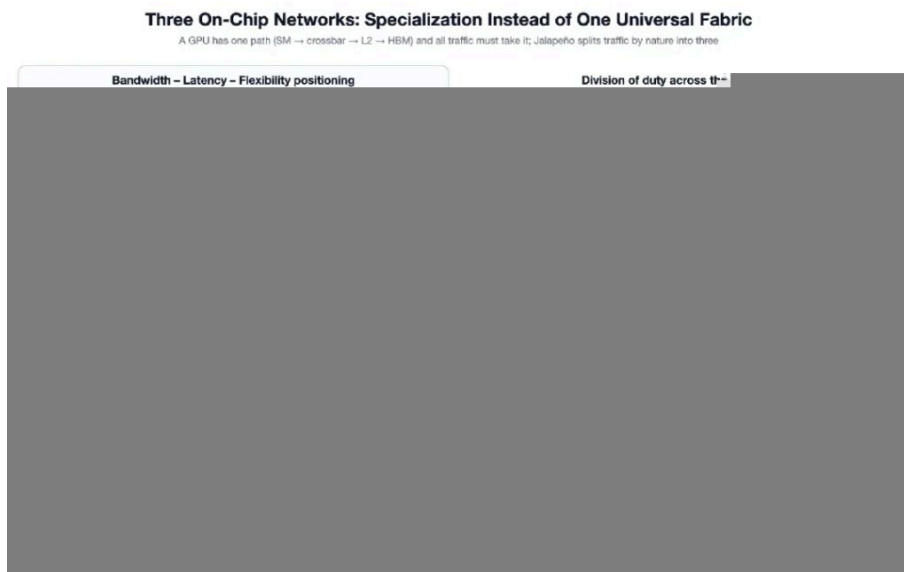
### 3.3 内存子系统

内存子系统最大的变化就是 NUMA 结构. 我们参考 HBM4 的一些数据, 单个 HBM4 stack 包含 64 个 pseudo-channel(PC) 位宽为 32bit, burst length = 8. 因此我们推算连接的结构如下

连接到单个 Core Slice 的位宽为 1536 bit, 每个 Core Slice 连接三颗 HBM, 总计带宽为 240GB/s, 我们可以看到这样的设计有一个好处, 比起 GPGPU 的统一内存访问, 这种做法可以降低大量的抖动, 并且延迟更低.

### 3.4 片上网络

整体来看, 每个 Core Slice 和它直接连接的 HBM 之间有一个 Local view, 它主要负责 Core 内部的一些 KVCache 切片和权重切片存储, 然后针对多个 Core 之间的 NUMA 结构, 整个 NOC 分为了两块, 一部分专门用于集合通信, 另一部分就是一个通用的NOC.



和GPU的最大区别就在此, GPU的编程模型决定了任意线程可以任意访问全局内存的地址, 这是 CUDA 生态决定的, 但是不同的内存访问速度差距非常大, 这些差距需要 L2 来填平, 然而 L2 由于物理约束, 在 Blackwell/Rubin 这样 Dual-Die 的架构中已经被分为 4 片, 它自身也存在很大的延迟差, 这样整个系统就被很大的访问延迟拖慢了. 而 Jalapeño 把这些访问内存距离的复杂性通过 NUMA 架构分离, 芯片更加简单, 而对于算子编写的复杂度交给了编译器, 而整个编译器在 LLM-Agent 的帮助下又可以进行极致的性能优化. 因此最终在芯片上, Jalapeño 获得了面积/功耗/延迟的三重收益.

我们推测 Genral NOC 是一个小带宽局部 Mesh + Global Switch 的架构, 而独特的是 Collective Network, 这里我们稍微展开分析一下, 考虑到延迟等约束, 我们仿真了几种拓扑结构, 最终认为它是一个 8x8 的 2-stage 架构, 每 8 个 Core Slice 一组, 然后全局在针对 8 组进行归并.

它需要支持 64 个 Core Slice 的 all-reduce 和 all-gather / broadcast 的能力, 并且直接连接到每个 Core Slice 的 L1. 整体上配合 Core Slice 超标量执行的结构, 它可以构建非常细粒度的 Tile based collective, 因此整体的延迟可以在 GEMM 计算时和其它算子 Fuse 的时候很好的被掩藏.

### 3.5 系统架构

整个系统和阿里的磐久AL128超节点类似的采用了 CPU-Tray和 Compute-Tray分离的架构, OpenAI 的CPU Tray(codename: Katsu) 为 2x Turin X86 CPU 1.5TB 内存, Compute Tray(codename: Vindaloo)包含 8 颗 Jalapeño ASIC, 两者之间通过 PCIe DAC 互联. 然后它采用了 2 套 ScaleUP 进行互联, 都使用了标准的 Ethernet 方案, 其中 Local Domain ScaleUP 支持128卡, 带宽为 600GB/s 主要用于 TP 并行, 而 Global domain 支持 2048 卡, 带宽为 200GB/s

Local Domain 采用背部的 Cable-Tray 用铜缆和 Switch-Tray(codename: Chana)互联, 而 Global Domain 则采用光互联. Switch-Tray 采用成熟的 BRCM Tomhawk 6 102.4T 以太网交换芯片. 它选择标准以太网, 相对于 Nvidia GPU 1.8TB/s ScaleUP带宽小了很多. 但是交换机直接买 Broadcom Tomahawk 6 标准件, 不需要自研 switch ASIC, 对一个 9 个月要流片的项目, 这省掉的不只是设计工作量, 更是一整条风险, 供应链与生态风险大幅降低. 这也是我在 2020 年就开始做 Ethernet ScaleUP(NetDAM)的出发点.

整个机柜架构如下右图所示:

左侧放置的是 CPU 服务器, 采用 2U 结构共计 16台, 中间的跨机柜的线缆就是PCIe DAC Cable , 每个 CPU 服务器累计 8 根, 估计采用的是 PCIe Gen5x8 的结构连接到 Compute-Tray, 或者在 Compute-Tray 内部有两颗 PCIe Switch 芯片, CPU 和 PCIe Switch 通过 4x PCIe-Gen5x16 连接. 右边上下各放置 8 个 Compute-Tray, 中间则是 6个用于 Local Domain 的1U Switch-Tray, 每个 Switch Tray 上放置 1 颗 TH6, 而用于 Global Domain 的采用的是 2U 的机框, SemiAnalysis 认为这里是包含 2 颗 TH6 的交换芯片, 但是如果是我来设计, 可能这个地方主要是放置一些 Shuffle box 或者一些小的 OCS / retimer 以及给光模块供电散热的组件即可, 而将 Global Domain 的交换机单独放置到一个机柜, 因为它是一个 Rail-based 拓扑.

整个设计的考虑是很充分的, 对于 TP 在 Local Domain 处理, 对于EP 使用更低带宽的 Global Domain. 并且对于 Jalapeño 而言, 并不需要像 Nvidia 那样攒够很大的 Batch 做 EP, 因此同步代价相对较少.

## 4. 软件架构

传统编译器的核心难点是在没有真实测量的情况下预测性能, 于是必须构造 cost model. Cost model 的精度决定了编译器的上限, 而在一个有 cache、有乱序、有片上网络竞争的机器上, cost model 的精度天花板很低. Jalapeño 的软件栈放弃了预测. 它把 mapping 决策变成一个可测量的搜索问题: 生成候选 → 在 chilisim 或真硅上跑 → 拿真实数字 → 迭代.

Architecture model: Simple for humans to understand

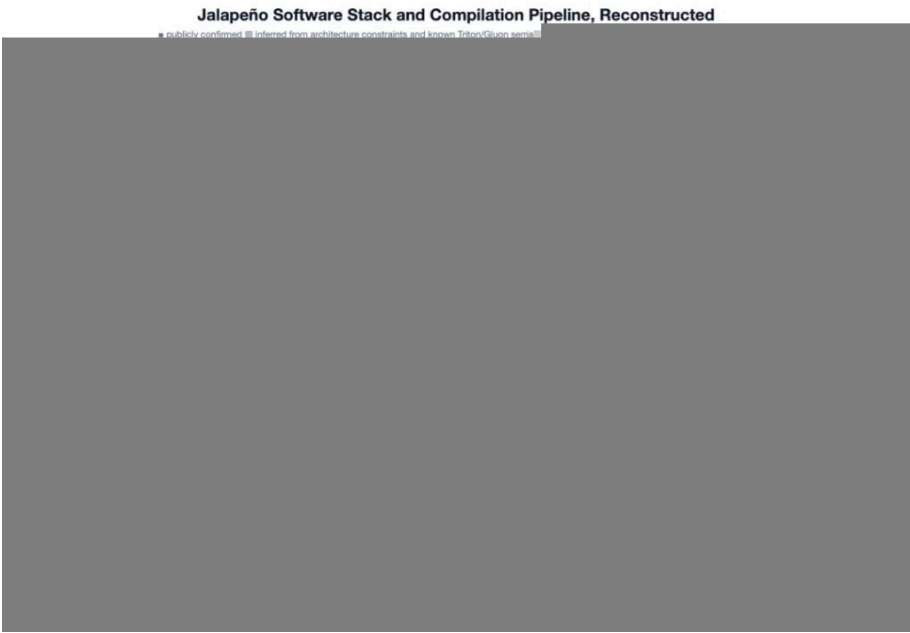


这是它和传统的 GPU 芯片最大的区别, 充分借助了现代 LLM-Agent 的能力来构建整个软件系统. 也就是 OpenAI 强调的一句话: *Spatial programming is onerous for humans. It is easy for frontier AI.*

整个系统的结构分为两层, Lowering 采用 Gluon 并 配合 Linear Layouts代数(详细分析可以参考 《学习一下 Linear Layout》). 然后 mapping 到硬件更多的使用了 LLM-Agent 进行搜索和调优.

然后整个公开的软件协议栈分为六层:

整个框架的详细过程总结如下:



这张图把 Jalapeño 的软件栈分层与本文推演的编译 Pass 流水线画在一起, 图例要先看: 实心方块是已公开确认, 斜线纹是由架构约束与 Triton/Gluon 已知语义推断, 紫色是由 AI 搜索而非 cost model 决定.

左侧分层 (Teacup / gigakernel / Gluon / Linear Layouts + TensorInfo / ~3000 行类汇编 kernel / chilisim) 每层都给了 CUDA 栈的近似对应 (vLLM ↔ Teacup, CUDA Graph ↔ gigakernel, CuTe ↔ Linear Layouts, PTX/SASS ↔ 类汇编, NCU ↔ chilisim), 但关键差异只有一条: CUDA

栈里由硬件调度器决定 CTA 落到哪个 SM, 这里由编译期 / AI 决定 program 落到哪个物理 core , 即静态空间映射取代动态调度.

右侧 P0-P7 的 Pass 流里, 标 ★ 的两个 (P2 slice 放置 / 亲和性分配, P4 tiling / 流水化 / 预取距离) 在 GPU 编译器里根本不存在: P2 决定每个权重分片、每条 KV、每个 expert 归属哪个 HBM slice , TensorInfo 就是承载这个决策的载体, 它同时编码逻辑 layout 与物理 placement; P4 则因为“硬件 L1 + 预取”没有可靠的解析 cost model (最优预取距离随 shape / 命中率 / OoO 窗口占用而变) 而天生适合交给搜索.

下方那条 DeepSeek MLA 的 40 小时实测曲线是全章最硬的证据: 功能正确 0.31% → FP8 矩阵 31.7% → 分块前瞻重缩放 59.2% → V-matmul 调度 77.1% → K-tile 预取 + 合并访存 88.9%, 五步依次对应传统编译器的指令选择 → 数值算法重排 → 指令调度 → 访存优化 — AI 干的是 auto-scheduler + auto-tuner 的活.

图末推论回答了“为什么可以放弃通用编译器”: 硬件只需保证上界足够高 + 语义足够清晰可搜索, 找到达到上界的那条路径就从“编译器研究问题”变成“搜索算力问题”.

这也是我为什么前段时间在尝试做 Anthropic 的那个 VLIW+SIMD 的面试题, 当然不是为了找工作, 而是为了更好的深入理解整个优化流程.

我也针对 Gluon 的代码使用一些 AI 工具进行详细的分析, 具体的分析内容就不公开了..下图是一个大概的流程分析

另一方面我们需要关注 Gluon 和 XLS 的配合, 这是软硬件协同设计的很好的参考, 下一章我们将详细展开分析

## 5. 芯片设计

整个项目从第一行RTL开始到流片只用了 9 个月, 基本上完全颠覆了现有的芯片开发流程, 这也是我最近很长一段时间非常关注的方向.

### 5.1 关键前提: 先换语言, 再让 AI 上场

整个设计的关键前提是, 先换语言, 再让 AI 上场

注意这一页的版式本身就是论点: AI + DESIGNER 与 Internal AI model、Fast tooling 三者都用箭头指向中间那个蓝色的 XLS HW language , 也就是说, 在 OpenAI 自己的叙述里, XLS 是 AI 与设计者共同作用的那个基座. 我们注意到 XLS对自己的定位是:

XLS is a 'Mid-Level' Synthesis toolchain for hardware development. It is similar and has some of the same goals as High-level Synthesis (HLS) tools however it operates at a generally lower level.

四条的共同点是: 让每一步都机器可判定. 这是把“设计”从一个需要人类判断的过程, 改造成一个可以自动评分的搜索问题. 推测整个过程如下:

## 5.2 设计如何闭环

OpenAI 在 Hotchips 的演讲资料如下:

其中最关键的是人机的分工, 人类负责定义一些目标函数, 和选择整个芯片的主体结构, 判断哪些架构复杂是否可以删掉, 并规划整个软硬件协同的宏观架构. 而 AI 负责在结构里进行详细的高频度的穷尽搜索, 把工程师从手工调整面积/时序的工作里解放出来. 分工的边界是 *让 AI 负责擅长的在明确判据的情况下进行大规模搜索*, 而不是在一些宏观的系统架构上进行没有明确目标评判的生成代码.

## 6. 未来展望

### 6.1 对比 Nvidia GPGPU架构

我们把它和 Nvidia GPGPU 做一个完整的对比:

GPU 的存储层次是 寄存器 → L1/SMEM → (Cluster/DSMEM) → 统一 L2 (50-126 MB) → HBM, 其中统一 L2 是全芯片共享的, 所有 SM 通过全局 crossbar 访问所有 HBM 通道.

另外非常值得对比的是它们的 Tensor Unit的架构

详细做了一个对比:

从芯片面积上来看, Jalapeño 使用了更多的 tensor unit, 估计面积在 180 mm<sup>2</sup> 占整个 Die 面积的 22%, 而没有使用很大的 L2 Cache 和复杂的 RF/SMEM/TMEM/L1Cache 的内存层次结构.

Jalapeño 把 core 和 HBM 切成 **64 个 slice**, 每个 core slice 对自己那份 HBM slice 有一个 **low-latency local view**; slice 之间的同步走**专用的高带宽集合通信网络**; 另有一张**通用 NoC** 负责杂活和通往 scale-up 网络. 官方把这个称为 “minimal memory hierarchy”, 并明确说相对 GPU 的复杂内存系统这是巨大优势.

另外整个系统设计代表了两种不同的思路, Nvidia GPGPU 通过更高的 Occupancy 和 Latency hiding 获得更高的吞吐, 对于训练场景是非常有效的, 但是对于推理小batch 并且有严格SLA需求的场景, 这种方式就不适合了. 而 Jalapeño 用“缩短距离 + 预取 + 乱序窗口”直接消除延迟, 不依赖并发度.

另一方面, Transformer 每一层里, attention 输出要跨 head 合并、MLP 输出要跨 hidden 维求和. 一旦这些维度被切到多个执行单元上 (片内 slice 或片间 TP), 每层就至少需要一次归约. 层数是模型结构给定的, 归约次数因此也是给定的:

第一项随并行度线性下降, 第二项不随并行度下降, 通常还随并行度上升, 因为要跨更远的域. 因此, 在 NOC 设计上也要考虑到这方面的影响, 如果能够显著的降低这方面的延迟开销, 那么我们就可以更有效的提高 TP 并行的规模.

两种架构的对比如下:

### 6.2 OpenAI roadmap

在 LLM-Agent 的协作下, OpenAI宣称整个芯片(A0)回片后经过进一步优化, B0估计还有25% 性能提升已经送出流片, 未来第二代将支持训练

### 6.3 一些分析总结

总体来看, 在今年早些时候我就注意到了 LLM-Agent 和芯片协同设计这个巨大的机会, 从使用 LLM-Agent 详细逆向分析 Blackwell 的架构, 到自己动手把 Anthropic VLIW+SIMD challenges 做到第三名, 基本上已经完全掌握了整个迭代流程, 几个月前芯片的架构基本上也整理清楚了, 基本判断和 OpenAI 是一致的, 这也正是去年我在分析 Blackwell 架构时, 老黄在公司内部转发的那篇文章中所提到的, 一定要把现有的 warp 调度机制改变才能在 Perf-Watt Ratio 上获得数倍的提升.

而对于互联, 本来就是我最擅长的, 从 2020 年开始全球领先的 Ethernet Based ScaleUP 实现, 到现在 DPU/NIC 领域全面超越 Nvidia, 下一代在 PPA 上 DPU/NIC 也会对Nvidia 有碾压的优势, 就像 Jalapeño 这样数倍到数十倍的提升. 届时无论是 ScaleUP/ScaleOut 能和我们掰掰手腕的也没几家, 而现在算力芯片基本也准备好怎么弄了...

本文来源: [zartbot](#)

#### 风险提示及免责条款

市场有风险, 投资需谨慎。本文不构成个人投资建议, 也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资, 责任自负。

 79    收藏

分享到:  

#### 写评论

请发表您的评论

 表情    图片

发表评论